

Mobil Geliştiriciler İçin OS

Ömer Faruk Yavuz

December 2025

Giriş

Mobil uygulama geliştirme, kullanıcı arayüzü akıcılığı, düşük gecikme, enerji verimliliği ve yüksek kararlılık gerektiren sınırlı bir çalışma ortamında yürütülmektedir. Bu nedenle Android ve iOS işletim sistemlerinin süreç (process), iş parçacığı (thread), zamanlama (scheduling) ve eşzamanlılık (concurrency) modellerinin doğru anlaşılması, bir mobil geliştiricinin performans, stabilite ve kullanıcı deneyimi açısından sağlıklı kararlar verebilmesi için kritik önem taşır.

Bu dokümanın amacı, mobil geliştirmede sıkça karşılaşılan jank, ANR, frame drop, UI donması, memory leak veya race condition gibi problemlerin temelinde yatan işletim sistemi davranışlarını teknik olarak açıklamaktır. Burada aktarılan kavramlar; React Native, Flutter, Swift/Objective-C, Kotlin/Java gibi tüm modern mobil geliştirme ortamlarında geçerli olup, farklı framework'lerin thread modellerini anlamayı ve doğru uygulamayı kolaylaştırır.

Dokümanın kapsamı şu başlıklar etrafında şekillendirilmiştir:

- Process ve thread mimarisi
- Bellek yönetimi, heap-stack ayrımı
- Event loop modelleri (JavaScript event loop, Android Looper, iOS Run-Loop)
- İşletim sistemi zamanlayıcıları ve öncelik seviyeleri
- Asenkron çalışma paradigmaları (Promises, async/await, coroutines)
- Mobilde uzun işlemlerin doğru yönetimi
- Native ve hibrit framework'lerde thread kullanım stratejileri

Bu bilgiler, mobil geliştiricilerin platform davranışlarını daha iyi kavrayarak uygulama mimarisini bilinçli şekilde tasarlamasını ve performans sorunlarını kökünden çözebilmesini hedeflemektedir.

Mobil Uygulama Mimarisinin Katmanlı Şeması

[UYGULAMA KODUN]

(React Native, Swift, Kotlin, Flutter vs.)

[UYGULAMA FRAMEWORK / RUNTIME]

- Android: Jetpack, Android SDK, Android Runtime (ART)
- iOS: UIKit / SwiftUI, Cocoa Touch
- RN: JS Engine (Hermes/JSCore) + Bridge
- Flutter: Flutter Engine, Skia

[İŞLETİM SİSTEMİ (USER SPACE)]

- Sistem servisleri:
 - Window Manager / SurfaceFlinger (Android)
 - SpringBoard (iOS)
 - Notification service
 - Location service
 - Media server (audio/video)
 - System daemons (network, time, logs, ...)

[KERNEL]

- Process & thread scheduler
- Memory management
- File system
- Network stack (TCP/IP)
- Security / permission enforcement
- System call arayüzü (syscall)

[DONANIM]

- CPU, GPU
- RAM
- Disk / flash storage
- Ekran, dokunmatik
- Kamera, mikrofon, sensörler (GPS, gyro, accel...)
- Modem (Wi-Fi / LTE / 5G)

User Space ile Kernel Arasındaki Etkileşim: Sistem Çağrıları

Modern işletim sistemlerinde uygulamalar doğrudan donanım kaynaklarına erişim sağlayamaz. Güvenlik, yalıtım (isolation) ve kararlılık gereksinimleri nedeniyle, uygulama kodu *user space* adı verilen sınırlı bir çalışma ortamında yürütülür. Donanıma veya çekirdek seviyesindeki servis bileşenlerine erişim, yalnızca *system call* (sistem çağrısı) adı verilen kontrollü bir mekanizma üzerinden gerçekleştirilir. Bu bölümde user space ile kernel space arasındaki etkileşim, adım adım açıklanmaktadır.

System Call Kavramı

Bir system call, kullanıcı modunda çalışan bir sürecin (ör. mobil bir uygulamanın) işletim sisteminden belirli bir hizmet talep etmesini sağlayan kontrollü bir geçiş mekanizmasıdır. Dosya açma, soket oluşturma, bellek tahsis etme veya zaman bilgisini okuma gibi işlemler, doğrudan donanıma erişim gerektirdiğinden kernel tarafından yürütülür. Dolayısıyla user space uygulamaları bu işlemleri doğrudan gerçekleştiremez; bunun yerine ilgili system call fonksiyonunu çağırır.

System Call Sürecinin Genel Akışı

User space ile kernel space arasındaki etkileşim aşağıdaki aşamalardan oluşur:

1. **Kullanıcı Kodunun Çağrısı:** Uygulama, bir kütüphane fonksiyonu (`open()`, `read()`, `send()` vb.) aracılığıyla bir system call'ı tetikler. Bu çağrı, user space içinde çalışan standart C kütüphanesi (Android'de *bionic libc*, iOS'ta *libSystem*) tarafından hazırlanır.
2. **CPU Mod Değişimi:** System call tetiklendiğinde işlemci, özel bir CPU komutu (ARM mimarisinde `svc`, x86-64 mimarisinde `syscall`) aracılığıyla *user mode* ortamından *kernel mode* ortamına geçiş yapar. Böylece kernel ayrıcalık seviyesine sahip olur ve donanım kaynaklarına erişebilir.
3. **Kernel System Call Handler'ının Çalışması:** İşletim sistemi, system call numarasına karşılık gelen işleyiciyi (`sys_open`, `sys_read`, `sys_write` vb.) çağırır. Bu işleyici, kernel alt sistemlerinden ilgili bileşene (örneğin dosya sistemi yöneticisi, bellek yöneticisi veya ağ yığını) isteği yönlendirir.
4. **Alt Sistemlerin İşlemi Gerçekleştirilmesi:** Kernel alt sistemleri gerekli işlemleri yürütür. Örneğin bir dosya açma isteği, önce sanal dosya sistemi (VFS) katmanından geçer ve ardından ilgili dosya sistemi sürücüsüne iletilir.
5. **Kernel'den User Space'e Dönüş:** İşlem tamamlandığında kernel, sonucu (başarı durum kodu, dosya tanımlayıcısı, hata kodu vb.) user space'e döndürür ve işlemci tekrar *user mode* seviyesine geçiş yapar.

Performans ve Gecikme Açısından System Call'lar

System call'lar, user space içinde gerçekleştirilen sıradan fonksiyon çağrılarına kıyasla daha yüksek maliyetlidir. Bunun temel nedenleri şunlardır:

- CPU mod değişimi (user mode $\rightarrow$ kernel mode),
- Kernel yığınlarının ve veri yapıların kurulması,
- Gerekli kernel alt sistemlerinin çalıştırılması,
- Donanım sürücülerinin tetiklenmesi ve olası kesmelerin (interrupt) işlenmesi.

Bu nedenle, dosya işlemleri, ağ iletişimi veya yüksek sayıda system call gerektiren işlemler kullanıcı arayüzü (UI) iş parçacığında çalıştırılmamalıdır. Aksi halde uygulamada gecikmeler, kare düşmeleri ve *ANR (Application Not Responding)* gibi sorunlar ortaya çıkabilir.

Güvenlik Açısından System Call Mekanizması

System call'lar aynı zamanda işletim sisteminin güvenlik politikasının uygulandığı kritik noktalardır. Kernel, her talebi gerçekleştirmeden önce:

- çağrıyı yapan sürecin kullanıcı kimliğini (UID/GID),
- süreç izinlerini,
- sandbox veya SELinux gibi ek güvenlik politikalarını

denetler. Bu sayede, uygulamaların izinsiz şekilde kamera, mikrofon, ağ arabirimi veya dosya sistemi gibi kaynaklara erişmesi engellenmiş olur.

Genel Değerlendirme

System call mekanizması, modern işletim sistemlerinin güvenli, yalıtılmış ve denetimli çalışmasını sağlayan temel yapı taşlarından biridir. Mobil uygulama geliştirme açısından bu mekanizmanın anlaşılması, performans optimizasyonu, gecikme analizi, izin yönetimi ve hata ayıklama gibi konularda doğrudan avantaj sağlar. Özellikle arayüz iş parçacığının bloklanmasının neden ANR veya kare düşmesine yol açtığı, bu etkileşim modeli incelendiğinde açık bir biçimde anlaşılmaktadır.

Çekirdek Zamanlayıcısı: İş Parçacıklarının Planlanması

Bir işletim sisteminin temel sorumluluklarından biri, sınırlı işlemci kaynaklarını (CPU çekirdekleri) birden fazla süreç ve iş parçacığı arasında etkin biçimde paylaşmaktır. Bu görevi yerine getiren bileşen *kernel scheduler* (çekirdek zamanlayıcısı) olarak adlandırılır. Scheduler, hangi iş parçacığının ne zaman ve ne kadar süreyle çalışacağını belirleyerek sistemin yanıt verebilirliğini, performansını ve adaletini (doğru kaynak paylaşımı) sağlar.

Zamanlayıcının Temel Görevleri

Bir kernel zamanlayıcısının temel fonksiyonları aşağıdaki şekilde özetlenebilir:

- **İş Parçacığı Seçimi:** Her CPU çekirdeğinde çalışmak üzere bir iş parçacığı (thread) seçer. Bu seçim, öncelik (priority), geçmiş çalışma süresi ve planlama politikasına göre yapılır.
- **Zaman Dilimi Tahsisi (Time Slicing):** Çoklu görev ortamlarında her iş parçacığına belirli uzunlukta bir zaman dilimi atanır. Bu süre dolduğunda scheduler devreye girerek başka bir iş parçacığı çalıştırabilir.
- **Adaletin Sağlanması (Fairness):** Yoğun CPU kullanan iş yüklerinin tüm sistemi bloke etmesini engeller; düşük öncelikli görevlerin de belirli aralıklarla çalışmasını garanti eder.
- **Yanıt Verebilirlik:** Özellikle kullanıcı arayüzü veya gerçek zamanlı iş yüklerinde, düşük gecikme (low latency) kritik olduğundan scheduler bu tür iş parçacıklarına daha yüksek öncelik tanıyabilir.
- **Bloklanmış ve Hazır Durumların Yönetimi:** I/O bekleyen veya kilit (lock) nedeniyle bloklanan iş parçacıklarının durumunu takip eder ve uygun an geldiğinde tekrar çalıştırılmak üzere hazır kuyruğuna alır.

Zamanlayıcı Kuyrukları ve Durum Geçişleri

Her iş parçacığı, çekirdek tarafından tanımlanan belirli durumlar arasında geçiş yapar. Zamanlayıcı, bu geçişleri takip ederek uygun anda işlemciyi tahsis eder.

- **Running (Çalışıyor):** İş parçacığı bir CPU çekirdeğinde aktif olarak çalışmaktadır.
- **Ready (Hazır):** Çalışmaya hazır durumdadır fakat CPU tahsisi beklemektedir.
- **Blocked (Bloklanmış):** I/O, kilit veya başka bir dış kaynağı beklemektedir; CPU kullanamaz.

- **Waiting (Bekliyor):** Zamanlayıcı tarafından belirli bir olayın veya zamanın gelmesi beklenmektedir.

Scheduler, yalnızca *running* ve *ready* durumlarındaki iş parçacıklarını CPU üzerinde planlar. Bloklanmış iş parçacıkları, gerekli olay gerçekleştiğinde tekrar hazır kuyruğuna alınır.

Planlama Politikaları

Modern işletim sistemlerinde birden fazla planlama politikası bulunur. Mobil işletim sistemleri (Android ve iOS), tipik olarak aşağıdaki yaklaşımları benimser:

- **Completely Fair Scheduler (CFS):** Linux tabanlı Android'de kullanılan varsayılan mekanizmadır. Tüm iş parçacıklarına hesaplanan oranda CPU süresi vererek adil bir çalışma sağlar. Kırmızı-siyah ağaç (red-black tree) tabanlı bir veri yapısı kullanır.
- **Gerçek Zamanlı (Real-Time) Planlama:** Ses işleme (audio), video oynatma veya düşük gecikme gerektiren senaryolarda daha yüksek öncelikli, kesintisiz çalışması gereken iş parçacıkları için kullanılır.
- **Background/Foreground Ayırımı:** Mobil ortamda kullanıcı arayüzünü temsil eden iş parçacıkları (foreground) düşük gecikme için öncelikli değerlendirilirken, arka plan görevleri (background) düşük öncelik alarak batarya ve performans dengesini korur.

Mobil Uygulamalar Açısından Önemi

Scheduler davranışının anlaşılması, mobil uygulama geliştirmede birçok performans sorununu açıklığa kavuşturur:

- **UI Thread Bloklanması:** Kullanıcı arayüzü iş parçacığı diğer CPU yoğun görevler nedeniyle planlanamazsa, uygulama gecikme, donma veya ANR hatası üretebilir.
- **Yoğun CPU Kullanımı:** Arka planda yüksek hesaplama yapan iş parçacıkları, foreground görevlerinin zamanında çalışmasını engelleyebilir.
- **Enerji Tüketimi:** Sürekli uyanan ve CPU talep eden iş parçacıkları scheduler tarafından sıkça çalıştırılır; bu durum batarya tüketimini artırır.
- **Çok Çekirdekli İşlemcilerde Dağıtım:** Kernel, iş parçacıklarını birden fazla çekirdeğe dağıtarak paralel yürütme sağlar; bu, performansı belirgin şekilde etkiler.

Sonuç

Kernel zamanlayıcısı, sistem performansını ve yanıt verebilirliğini doğrudan belirleyen temel bileşendir. Scheduler'ın nasıl çalıştığını anlamak, mobil uygulamalarda jank, gecikme, ANR, fazla CPU kullanımı ve enerji tüketimi gibi kritik problemlerin kök nedenlerini doğru yorumlamak için gereklidir. Bu bakış açısı, uygulama seviyesindeki optimizasyonların altında yatan çekirdek davranışlarının anlaşılmasını sağlar.

Process ve Thread Temelleri

Mobil işletim sistemlerinde bir uygulama, *process* adı verilen yalıtılmış bir çalışma ortamında yürütülür. Her process, kendisine ait adres alanına, dosya tanımlayıcılarına ve işletim sistemi tarafından atanan kimlik bilgilerine sahiptir. Bir process içerisinde bir veya birden fazla *thread* (iş parçacığı) bulunabilir. Thread'ler aynı adres alanını paylaşır, bu nedenle veri paylaşımı hızlı olmakla birlikte yarış durumları (race condition) ve senkronizasyon hataları oluşabilir.

UI / Main Thread

Mobil işletim sistemlerinde kullanıcı arayüzü işlemleri özel bir iş parçacığında yürütülür; bu iş parçacığı *UI thread* veya *main thread* olarak adlandırılır. Ekranın çizilmesi, dokunma olaylarının işlenmesi ve arayüz güncellemeleri yalnızca bu thread üzerinden gerçekleştirilebilir.

Neden Önemlidir?

UI thread'in bloklanması, kare düşüşü (jank), donma (freeze) ve Android'de *ANR (Application Not Responding)* hatalarına yol açabilir. Ağ istekleri, dosya işlemleri veya ağır hesaplamalar gibi zaman alan işlemler UI thread'de yürütüldüğünde kullanıcı deneyimi doğrudan olumsuz etkilenir.

Mobil Geliştirici İçin Çıkarımlar

- Ağ istekleri, disk erişimi ve büyük veri işleme gibi pahalı operasyonlar her zaman arka plan thread'lerinde yürütülmelidir.
- UI güncellemeleri yalnızca main thread üzerinden yapılmalıdır.
- Uzun süren işlemleri main thread'e taşımamak için asenkron yapılar (Android'de coroutines, iOS'ta GCD ve async/await, React Native'de asenkron JS API'leri) kullanılmalıdır.
- Aynı hafıza alanını paylaşan thread'ler arasında senkronizasyon mekanizmalarına dikkat edilmelidir.

Sonuç

Process ve thread kavramlarının anlaşılması, mobil uygulamalarda performans sorunlarının kök nedenlerini görmeyi mümkün kılar. Özellikle UI thread'in bloklanmaması, akıcı ve kullanıcı dostu bir deneyim sağlamak açısından kritik öneme sahiptir.

Bellek Yönetimi (Memory Management)

Mobil cihazlarda bellek miktarı sınırlıdır ve uygulamalar belirli bir RAM tahsis alanı içinde çalışmak zorundadır. İşletim sistemi, her uygulamaya ayrı bir sanal bellek alanı sağlar; ancak bu alan fiziksel bellek üzerinde paylaşımlı olarak yönetilir. Uygulamalar, büyük veri yapıları, yüksek çözünürlüklü görseller veya gereksiz referanslar nedeniyle bellek kullanımını kontrol edemediklerinde sistem performans kaybı yaşayabilir veya uygulama işletim sistemi tarafından sonlandırılabilir.

Heap ve Stack Ayrımı

Uygulama belleği, çalışma zamanı davranışını etkileyen iki temel bölgeye ayrılır: *stack* ve *heap*. Bu iki bellek alanı farklı amaçlarla kullanılır, farklı performans özelliklerine sahiptir ve mobil uygulama geliştirme açısından farklı riskler doğurur.

Stack Belleği

Stack, fonksiyon çağrılarının sırayla yığıldığı (push/pop) hızlı bir bellek bölgesidir. Aşağıdaki özelliklere sahiptir:

- Fonksiyon çağrıları sırasında otomatik olarak büyür ve fonksiyon tamamlandığında otomatik olarak küçülür.
- Yerel (local) değişkenler ve fonksiyon parametreleri bu alanda saklanır.
- Bellek tahsisi ve serbest bırakılması işletim sistemi veya çalışma zamanı tarafından otomatik yönetildiğinden çok hızlıdır.
- Boyutu sınırlıdır; büyük veri yapılarının burada saklanması *stack overflow* hatasına yol açabilir.

Örnek (Basit):

```
function sum(a, b) {  
  let c = a + b;  // 'c' değişkeni stack'te saklanır  
  return c;  
}
```

Örnek (Mobil Geliştirici İçin): React Native veya Android Kotlin'de küçük karşılaştırma değişkenleri, sayısal değerler ve fonksiyon parametreleri stack üzerinde tutulur:

```
// Kotlin örneği  
fun calculateDistance(x: Int, y: Int): Int {  
  val diff = x - y  // 'diff' stack üzerinde  
  return diff  
}
```

Bu tür değişkenler fonksiyon bittiğinde otomatik olarak yok olur.

Heap Belleği

Heap, çalışma zamanında dinamik olarak oluşturulan nesnelerin saklandığı esnek bir bellek bölgesidir. Aşağıdaki özelliklere sahiptir:

- Boyutu stack'e göre çok daha büyük fakat erişim süresi daha yavaştır.
- Nesneler *new*, *Object()*, *class* örnekleri gibi dinamik oluşturma işlemleriyle burada tutulur.
- Bellek serbest bırakılması otomatik değildir; Android'de *Garbage Collector (GC)*, iOS'ta ise *ARC* (Automatic Reference Counting) tarafından yönetilir.
- Yanlış referans yönetimi, *memory leak* oluşumuna neden olabilir.

Örnek (Basit):

```
let user = {  
  name: "Omer",  
  age: 28  
};  
// 'user' nesnesi heap üzerinde saklanır
```

Örnek (Mobil Geliştirici İçin — React Native):

```
const [items, setItems] = useState([]);  
// 'items' dizisi heap'tedir  
// doğru yönetilmezse bileşen kapandıktan sonra bile RAM'de kalabilir
```

Örnek (Android Kotlin):

```
data class User(val name: String, val age: Int)  
  
val currentUser = User("Omer", 28)  
// currentUser nesnesi heap üzerinde tutulur
```

Heap ve Stack'in Karşılaştırılması

- **Stack hızlıdır**, çünkü tahsis ve serbest bırakma işlemleri deterministiktir (fonksiyon çağrı sırasına göre).
- **Heap daha yavaştır**, çünkü bellek yönetimi karmaşıktır ve GC/ARC gibi mekanizmalar devreye girer.
- Stack kısa ömürlü veriler içindir, heap uzun ömürlü nesneleri saklar.
- Heap aşırı büyürse uygulama *Out of Memory* ile kapanabilir.

Mobil Geliştirici İçin Pratik Sonuçlar

- Büyük veri yapıları (Array, Map, görseller) heap'te tutulduğu için performansı etkiler.
- Gereksiz referanslar temizlenmezse heap büyür ve GC baskısı oluşur, bu da jank (kare düşmesi) yaratır.
- UI thread üzerinde heap tahsisi yapmak (özellikle büyük nesneleri oluşturmak) kısa süreli donmalara yol açabilir.
- Stack üzerinde büyük veri yapıları oluşturulamaz; bu nedenle büyük resimler, JSON'lar veya diziler her zaman heap üzerinde yer alır.

Sonuç

Stack ve heap ayrımını anlamak, mobil uygulamaların bellek kullanımını doğru analiz etmek için gereklidir. Yanlış yönetilen heap nesneleri performans düşüşü, bellek sızıntısı ve uygulama çökmesine neden olabilirken, stack'in sınırlı yapısı büyük veri yapılarının burada tutulmasının imkânsız olduğunu gösterir. Bu ayrımın bilinmesi, performans odaklı mobil gelişimde temel bir gerekliliktir.

Memory Leak Kavramı

Memory leak, uygulamanın artık ihtiyaç duymadığı nesnelere referans tutmaya devam etmesi sonucunda, bu nesnelerin çöp toplayıcı (GC) veya otomatik referans sayımı (ARC) tarafından serbest bırakılmaması anlamına gelir. Leak birikimi, uygulamanın RAM kullanımının sürekli artmasına ve nihayetinde işletim sisteminin uygulamayı sonlandırmasına neden olabilir.

Out of Memory (OOM) Durumu

Bir uygulama, işletim sistemi tarafından belirlenen bellek sınırını aşarsa *Out of Memory (OOM)* hatası oluşur. Android ve iOS, düşük bellek durumunda uygulamayı arka planda veya ön planda sonlandırabilir. Bu durum genellikle büyük boyutlu resimler, sıkıştırılmamış veri yapıları veya kontrolsüz caching stratejileri sonucunda ortaya çıkar.

Mobil Geliştirici İçin Çıkarımlar

- Yüksek çözünürlüklü görseller uygun biçimde yeniden boyutlandırılmalı ve hafızaya gereksiz yük bindirilmemelidir.
- React Native, Android veya iOS projelerinde gereksiz referans tutmamak için bileşen yaşam döngülerine dikkat edilmelidir.
- Cache mekanizmaları kontrollü ve sınırlandırılmış şekilde kullanılmalıdır.

- Büyük JSON nesneleri ana iş parçacığında parse edilmemeli, gerekiyorsa streaming teknikleri kullanılmalıdır.
- Düşük bellek uyarıları (low memory warnings) uygun şekilde ele alınmalı ve bellek temizliği yapılmalıdır.

Sonuç

Bellek yönetiminin temellerini anlamak, mobil uygulamaların kararlılığı ve performansı için kritik öneme sahiptir. Yetersiz bellek yönetimi, kullanıcı deneyimini doğrudan etkileyen donmalar, gecikmeler ve çökmelerle sonuçlanır. Bu nedenle bellek kullanımının ölçülmesi, analiz edilmesi ve optimize edilmesi her mobil geliştiricinin temel sorumlulukları arasındadır.

Dosya Sistemi ve Depolama (Storage) Temelleri

Mobil işletim sistemleri, uygulamaların veri depolayabilmesi için yalıtılmış bir dosya sistemi alanı sağlar. Bu yapı, hem güvenlik hem de performans açısından belirli sınırlamalara sahiptir. Dosya okuma ve yazma işlemleri (*disk I/O*), bellek erişimine kıyasla çok daha yavaş gerçekleşir ve yanlış kullanıldığında kullanıcı arayüzünde gecikmelere veya donmalara yol açabilir.

Uygulama Sandbox Mimarisi

Her mobil uygulama, işletim sistemi tarafından kendisine ayrılmış özel bir dizin içinde çalışır. Bu mimari *sandbox* olarak adlandırılır ve uygulamaların birbirlerinin dosyalarına veya verilerine erişmesini engeller.

Android Sandbox Yapısı:

- `/data/data/<paket-adı>/`
- Uygulamanın dahili verileri, ayar dosyaları, veritabanları burada saklanır.

iOS Sandbox Yapısı:

- `AppName/Documents/`
- `AppName/Library/`
- `AppName/tmp/`

Bu yapı, uygulama verilerinin sistemde güvenli ve izole bir biçimde saklanmasını sağlar.

Dahili ve Harici Depolama

Android: Android işletim sisteminde iki temel depolama alanı bulunur:

- **Dahili Depolama (Internal Storage):** Yalnızca uygulamanın erişebileceği güvenli alandır. Buradaki dosyalar uygulama kaldırıldığında silinir.
- **Harici Depolama (External Storage):** Kullanıcının görebildiği ortak alandır (Resimler, Video, Downloads). İzin gerektirebilir.

iOS: iOS, harici depolama kavramına sahip değildir. Tüm depolama alanı uygulama sandbox'ı içerisinde organize edilir, ancak dosyalar farklı klasör politikalarına göre sınıflandırılır (kalıcı, cache, geçici vb.).

Disk I/O'nun Yavaşlığı

Disk erişimi, RAM erişimine kıyasla çok daha yavaştır çünkü fiziksel donanım üzerinde çalışma gerektirir. Bu nedenle:

- Büyük dosyaları okumak,
- JSON veya medya içeriği yüklemek,
- Log veya rapor dosyası yazmak,

gibi işlemler doğrudan kullanıcı arayüzü iş parçacığında yürütülmemelidir.

Örnek (React Native):

```
const data = readFileSync("large.json");  
// UI thread'i bloklar ve donmaya neden olur
```

Örnek (Android Kotlin):

```
val text = File("large.txt").readText()  
// Bu çağrı UI thread'de yapılırsa ANR riski oluşur
```

Depolama ile İlgili Mobil Geliştirici Çıkarımları

- Tüm disk işlemleri mutlaka arka plan iş parçacıklarında gerçekleştirilmelidir.
- Cache dosyaları kontrollü şekilde saklanmalı ve gerektiğinde temizlenmelidir.
- Kullanıcıya görünür alanlara (galeri, indirme klasörü) erişim için izin politikalarına uyulmalıdır.
- Büyük JSON, görüntü veya medya dosyaları bellekte değil streaming teknikleri ile işlenmelidir.
- Dosya sistemi operasyonları bloklayıcı olduğundan UI thread üzerinde çağrılmamalıdır.

Sonuç

Dosya sistemi ve depolama mekanizmalarının temel prensiplerini anlamak, mobil uygulamaların hızlı, duyarlı ve güvenilir çalışması açısından kritiktir. Disk erişiminin yüksek gecikme maliyeti, bu işlemlerin uygun thread'lerde yönetilmesini zorunlu kılar. Ayrıca sandbox mimarisi, uygulamanın güvenli bir çalışma ortamı içinde kalmasını sağlayarak veri izolasyonunu garanti eder.

iOS ve Android’de Harici Depolama Kavramının Karşılaştırılması

Mobil işletim sistemleri, uygulamaların dosya sistemi üzerinde nasıl ve nerede veri depolayabileceğini farklı güvenlik modelleri ile belirler. Android, geleneksel olarak *external storage* kavramını desteklerken, iOS bu mimariyi benimsememektedir. Bu bölümde her iki yaklaşım arasındaki farklar açıklanmaktadır.

Android’de External Storage

Android işletim sisteminde *external storage*, tüm uygulamaların erişebildiği ortak bir dosya sistemi alanı olarak tanımlanır. Bu alan:

- Kullanıcının doğrudan görebildiği (örn. DCIM, Downloads),
- Uygulamaların izin verildiği sürece okuyup yazabildiği,
- USB ile bilgisayara bağlandığında görünür hâle gelen,
- Uygulama sandbox’ı dışında konumlanan

bir depolama mimarisi sunar. Genellikle `/storage/emulated/0` veya `/sdcard` dizinleri üzerinden temsil edilir.

iOS’ta External Storage Kavramının Bulunmayışı

iOS, Android’deki anlamıyla tüm uygulamaların ortak erişimine açık bir “external storage” alanı sunmaz. Bunun temel nedenleri şunlardır:

- Her uygulama katı bir *sandbox* içerisinde çalışır ve diğer uygulamaların dosyalarına erişemez.
- Uygulama verileri yalnızca kendi **Documents**, **Library** ve **tmp** dizinleri içerisinde saklanabilir.
- iOS, kullanıcının cihazını USB ile bağladığında yalnızca fotoğraflara erişime izin verir; uygulama verileri görüntülenmez.

Dolayısıyla iOS’ta Android’deki anlamıyla ortak, serbestçe erişilebilir bir dosya alanı bulunmamaktadır.

iOS’ta Kullanıcı Aracılığıyla Erişilebilen Alanlar

Her ne kadar iOS global bir external storage mimarisi sunmasa da, kullanıcı aracılığıyla erişilebilen bazı depolama yöntemleri mevcuttur:

- **Files (Dosyalar) Uygulaması:** Uygulama yalnızca kullanıcı bir dosya seçtiğinde bu alana erişebilir. Uygulama doğrudan bu alana yazamaz veya okuyamaz.
- **iCloud Drive:** Uygulama, yine kullanıcı onayı ve seçim mekanizması aracılığıyla iCloud Drive üzerinden dosya alabilir veya kaydedebilir.

- **UIDocumentPickerViewController / File Picker:** Uygulamanın diğer uygulamaların depolama alanlarına doğrudan erişmesine değil, yalnızca kullanıcı tarafından seçilen dosyalarla çalışmasına izin verir.

Bu yapılar, iOS’un güvenlik ve gizlilik önceliklerine uygun bir biçimde sadece kullanıcı etkileşimiyle erişilebilen sınırlı bir dış depolama modeli oluşturur.

Sonuç

Android, sistem genelinde paylaşılan geniş bir external storage alanına sahip iken iOS, güvenlik ve yalıtım gereksinimleri doğrultusunda böyle bir modeli desteklemez. iOS’ta depolama yalnızca uygulama sandbox’ı içinde veya kullanıcı aracılığıyla erişilebilen özel alanlarda gerçekleştirilebilir. Bu nedenle iOS, Android’deki anlamıyla bir “external storage” mimarisine sahip değildir.

İzin Sistemi (Permission Management)

Mobil işletim sistemlerinde kamera, mikrofon, konum, rehber veya fotoğraf galerisi gibi kullanıcıya ait hassas verilere erişim, sıkı güvenlik politikaları ile kontrol edilir. Bu kaynaklara erişim, uygulamanın yalnızca kullanıcıdan açık izin alması durumunda mümkün olur. Modern izin yönetimi, hem Android hem de iOS tarafından zorunlu kılınmış olup mobil uygulamaların güvenliğini ve kullanıcı gizliliğini korumayı amaçlar.

İzinlerin Temel Amacı

İzin sisteminin başlıca hedefleri şunlardır:

- Uygulamaların hassas donanım ve verilere yetkisiz erişimini engellemek,
- Kullanıcıya hangi kaynaklara erişildiği konusunda şeffaflık sağlamak,
- Uygulamaları sandbox sınırları içinde tutarak güvenlik açıklarını azaltmak,
- Kullanıcı verisini korumak ve gizlilik standartlarını yükseltmek.

Android İzin Modeli

Android işletim sistemi, *runtime permission* mantığı ile çalışır. Buna göre:

- Tehlikeli (dangerous) kategoride yer alan izinler uygulama çalışırken kullanıcıdan talep edilir (örn. kamera, mikrofon, konum, rehber).
- Kullanıcı izni reddedebilir, tekrar sorulmasını engelleyebilir veya yalnızca uygulama kullanımdayken izin verebilir.
- Android 10 sonrasında arka plan konum erişimi ek bir özel izin gerektirmektedir.

Android Manifest Bildirimi:

```
<uses-permission android:name="android.permission.CAMERA" />
```

Android Runtime Örneği:

```
// Kotlin
ActivityCompat.requestPermissions(
    this,
    arrayOf(Manifest.permission.CAMERA),
    1001
)
```

iOS İzin Modeli

iOS, gizlilik odaklı daha sıkı bir izin modeline sahiptir. Temel prensipler:

- Her izin için Info.plist dosyasında açıklama eklenmesi zorunludur.
- Kullanıcı ilk çağrıda sistemi tarafından gösterilen bir diyalog üzerinden karar verir.
- Kullanıcı bir izni reddederse uygulama tekrar sorgulama yapamaz, yalnızca Ayarlar uygulamasına yönlendirebilir.
- İzin verilmediğinde ilgili API çağrıları başarısız olur ve null/empty değer döner.

iOS Info.plist Örneği:

```
<key>NSCameraUsageDescription</key>
<string>Kamera erişimi gerekmektedir.</string>
```

Swift Örneği:

```
AVCaptureDevice.requestAccess(for: .video) { granted in
    // granted == true / false
}
```

İzin Verilmediğinde OS'nin Davranışı

Bir kullanıcı izin vermediğinde işletim sistemi ilgili kaynağa erişimi engeller. Bu durumda:

- Kamera açma isteği başarısız olur,
- Mikrofon kaydı başlatılamaz,
- Konum verisi boş veya hatalı gelir,
- Galeri erişimi kısıtlanır,
- Bildirim gönderimi kullanıcı onayına bağlıdır.

Uygulama bu senaryoları doğru şekilde ele almazsa beklenmeyen hatalar, boş ekranlar veya çökmeler meydana gelebilir.

Mobil Geliştirici İçin Pratik Sonuçlar

- Kullanıcı izin vermediğinde uygulamanın vereceği tepki mutlaka belirlenmelidir (fallback UI, uyarı, yönlendirme vb.).
- İzin istemeden önce kullanıcının neden bu iznin gerekli olduğunu anlayabilmesi için açıklayıcı ekranlar sunulmalıdır.
- Her iznin iOS ve Android'deki davranışı farklı olduğundan çapraz platform projelerde platforma özgü kontrol yapılmalıdır.
- Arka plan konum, mikrofon veya Bluetooth gibi özel izinlerin ek kısıtlamaları dikkate alınmalıdır.

Sonuç

İzin sistemi, mobil uygulamaların güvenlik modelinin temel bileşenlerinden biridir. Uygulamanın donanım kaynaklarına erişebilmesi, kullanıcı onayına ve işletim sisteminin politikalarına bağlıdır. Bu nedenle izin yönetimine dair temel prensiplerin bilinmesi, hatasız ve kullanıcı dostu bir uygulama geliştirmek için zorunludur.

Arka Plan Çalışma Kısıtlamaları (Background Limitations)

Mobil işletim sistemleri enerji tüketimini, güvenliği ve kullanıcı deneyimini optimize etmek amacıyla uygulamaların arka planda çalışma yetkilerini sınırlandırır. Bu kısıtlamalar, uygulamaların kontrol dışı şekilde sistem kaynaklarını tüketmesini engeller. Android ve iOS, arka plan çalışma politikalarını farklı mekanizmalarla uygular; ancak temel amaç her iki platformda da aynıdır: pil tasarrufu, performans dengesi ve kullanıcı kontrolü.

Arka Plan Çalışmanın Türleri

Bir uygulamanın arka planda çalışma ihtiyacı genel olarak üç kategoriye ayrılır:

- **Zamanlanmış görevler:** Belirli aralıklarla veri senkronizasyonu veya bakım işlemleri.
- **Sürekli çalışan görevler:** Müzik çalma, konum takibi, sağlık ölçer vb.
- **Kısa süreli arka plan işler:** Bir ekran kapandıktan hemen sonra yürütülen hızlı işlemler, örneğin küçük bir dosya yükleme.

Her mobil işletim sistemi bu kategorilere farklı seviyelerde izin verir.

Android’de Arka Plan Kısıtlamaları

Android işletim sistemi, özellikle sürüm 6.0 (Marshmallow) sonrasında pil verimliliğini artırmak amacıyla arka planda çalışan uygulamalara yönelik kapsamlı kısıtlamalar getirmiştir. Bu kısıtlamalar, geliştiricilerin arka plan işlerini doğru planlama araçlarıyla yürütmesini zorunlu kılar. Bu bölüm Android’in temel arka plan mekanizmalarını, davranışlarını ve uygulama geliştirme açısından sonuçlarını detaylandırmaktadır.

Doze Mode

Android 6.0 ile tanımlanan *Doze Mode*, cihaz uzun süre hareketsiz durumdayken enerji tüketimini azaltmak için arka plan aktivitelerini kısıtlar. Cihaz belirli koşullarda Doze durumuna girer:

- Ekran kapalı,
- Cihaz uzun süre sabit bir yüzeyde hareketsiz,
- Şarj edilmemekte,
- Kullanıcı etkileşimi bulunmamakta.

Doze modunda aşağıdaki davranışlar geçerlidir:

- Zamanlayıcılar (AlarmManager) ertelenir.
- Ağ erişimi büyük ölçüde engellenir.
- Arka plan servisleri çalıştırılmaz.
- Uygulamalar yalnızca *maintenance window* denilen kısa aralıklarda sınırlı işlem yapabilir.

Örnek: Doze Problemini Gidermek İçin WorkManager Kullanımı

```
// Android Doze Mode'da bile çalışan güvenilir bir görev
val request = PeriodicWorkRequest.Builder(
    SyncWorker::class.java,
    15, TimeUnit.MINUTES
).build()
```

```
WorkManager.getInstance(context).enqueue(request)
```

Bu yöntem Doze Mode tarafından bastırılmayan az sayıdaki mekanizmadan biridir.

App Standby

Android, uzun süre kullanılmayan uygulamaları *standby* durumuna alır. Bu durumda:

- Uygulamaya arka plan işlemleri için çok sınırlı kaynak ayrılır.
- Ağ erişimi geciktirilir veya tamamen engellenir.
- Zamanlanmış işler bekletilebilir.

Standby kategorisine giren uygulamalar genellikle:

- Uzun süredir açılmamış,
- Kullanıcı etkileşimi almamış,
- Bildirim almayan veya üretmeyen

uygulamalardır.

Background Execution Limits (Android 8.0+)

Android 8.0 (Oreo) ile birlikte arka plan servisleri (background services) çok daha sıkı şekilde kısıtlanmıştır. Uygulama arka plana geçtiğinde:

- Servisler yalnızca kısa bir süre (genellikle birkaç dakika) çalışabilir.
- Süre dolunca işletim sistemi servisi otomatik olarak sonlandırır.
- Uzun süreli görevler için foreground service zorunlu hâle gelir.

Hatalı Kullanım Örneği (Çalışmaz):

```
// Android 8.0+ cihazlarda kısa sürede öldürülür
startService(Intent(this, LongRunningService::class.java))
```

Doğru Kullanım (Foreground Service):

```
val intent = Intent(this, LocationService::class.java)
ContextCompat.startForegroundService(this, intent)
```

Foreground service, bildirim göstermek zorundadır.

WorkManager

WorkManager, Android'in enerji dostu ve OS uyumlu bir görev planlama API'sidir. Şu senaryolar için uygundur:

- Aralıklı senkronizasyon,
- Tek seferlik ertelenmiş işler,
- Ağ bağlantısı gerektiren görevler,
- OS tarafından yeniden başlatılsa dahi devam etmesi gereken işler.

Örnek:

```
class SyncWorker(context: Context, params: WorkerParameters)
: Worker(context, params) {
    override fun doWork(): Result {
        // Ağ isteği, dosya yükleme, veri senkronu vb.
        return Result.success()
    }
}
```

Foreground Service Requirement

Uzun süreli veya kritik işler (bildirimli çalışan servisler) için foreground service zorunludur:

- GPS konum takibi,
- Sağlık/fitness sensör takibi,
- Müzik oynatma,
- VOIP uygulamaları,
- Dosya yükleme/indirme.

Foreground service, kullanıcıya sürekli bir bildirim gösterir ve bu sayede Android işletim sistemi servisi öldürmez.

Örnek Bildirimli Servis:

```
val notification = NotificationCompat.Builder(this, "channel")
    .setContentTitle("Konum Takibi")
    .setContentText("Arka planda çalışıyor")
    .build()

startForeground(1, notification)
```

Sonuç

Android'in arka plan kısıtlamaları, geliştiricilerin servis, zamanlayıcı ve senkronizasyon işlemlerini rastgele tetikleyemeyeceğini gösterir. Doze Mode, App Standby, execution limits ve foreground service zorunlulukları dikkate alınmadan geliştirilen uygulamalar beklenmedik gecikmeler, kesintiler ve işlem iptalleriyle karşılaşabilir. Bu nedenle arka plan işlerinin modern Android mimarisine uygun olarak WorkManager veya foreground servislerle tasarlanması zorunludur.

Android İçin Kısa Örnek:

```
// Arka planda güvenilir bir işlem
WorkManager.getInstance(context).enqueue(...)
```

Bu mekanizma, Doze Mode tarafından kesintiye uğramayan az sayıdaki API'den biridir.

iOS'ta Arka Plan Kısıtlamaları

iOS, arka plan aktiviteleri konusunda Android'e göre daha katı bir mimariye sahiptir. Temel ilkeler:

- Bir uygulama arka plana geçtiğinde çoğu zaman “suspended” durumuna alınır; CPU kullanımı durdurulur.
- Arka planda yürütülebilecek işler yalnızca belirli kategorilere izinlidir (ör. müzik çalma, GPS, VoIP, Bluetooth, dosya yükleme).
- Geliştirici, bu izinlerin her biri için Info.plist dosyasında özel bildirim yapmak zorundadır.
- Zamanlanmış işler için `BGTaskScheduler` kullanılmalıdır.
- iOS, animasyonlar ve UI işlemleri dışındaki görevleri mümkün olan en kısa sürede sonlandırarak pil tüketimini sınırlamayı hedefler.

BGTaskScheduler Örneği:

```
// iOS'ta zamanlanmış arka plan görevi
BGTaskScheduler.shared.register(...)
```

Mobil Geliştirici İçin Pratik Sonuçlar

- Zamanlanmış işler için platformun kendi mekanizmaları (Android’de Work-Manager, iOS’ta BGTaskScheduler) kullanılmalıdır.
- Sürekli çalışan işler için foreground service (Android) veya izinli background mode (iOS) gereklidir.
- Uygulama ekranı kapandığında, geleneksel döngüler (`setInterval`, sonsuz loop vb.) işletim sistemi tarafından durdurulabilir.
- Pil tüketimi yüksek olan fonksiyonlar, sık tekrarlanan zamanlayıcılar ve ağ istekleri OS tarafından bastırılabilir.
- Uygulamanın arka planda ne kadar süre çalışabileceği platforma göre farklılık gösterir ve bu süre geliştirici kontrolünde değildir.

Sonuç

Arka plan çalışma politikaları, modern mobil işletim sistemlerinde pil tasarrufu ve kaynak yönetimi açısından kritik rol oynar. Uygulamaların arka planda sınırsız şekilde işlem yapması mümkün değildir ve bu kısıtlamaların bilinmesi işlevsel, güvenilir ve enerji dostu uygulamalar geliştirmek için zorunludur.

Android’de Arka Plan Çalışma Prensipleri ve Bildirim Zamanlama

Android işletim sistemi, gizlilik, pil verimliliği ve kullanıcı kontrolünü korumak amacıyla uzun süre arka planda çalışan işlemleri sıkı biçimde denetler. Bu nedenle geliştiricilerin arka plan görevlerini iki ayrı kategoride değerlendirmesi gerekir:

1. **Sürekli çalışan işler (long-running background tasks)**
2. **Zamanlanmış bildirimler (scheduled notifications)**

Bu iki kategori Android tarafında tamamıyla farklı mekanizmalara sahiptir. Bu bölümde her iki yaklaşım ayrıntılı olarak açıklanmaktadır.

Sürekli Çalışan İşler ve Foreground Service Gereksinimi

Android 8.0 (Oreo) sonrası, arka planda uzun süre çalışan servisler birkaç dakika içinde işletim sistemi tarafından sonlandırılır. Bunun temel nedeni, kullanıcıdan habersiz şekilde:

- konum takibi,
- mikrofon kullanımı,
- sensör izleme,
- sürekli ağ erişimi,
- yüksek pil tüketimi,

gibi işlemlerin yürütülmesini engellemektir.

Bu tür uzun süreli görevlerin yürütülmesi için tek istisna **foreground service** yapısıdır. Foreground service, kullanıcıya sürekli bir bildirim göstermek zorundadır; bildirim olmadan Android bu servisleri öldürür. Bunun nedeni, kullanıcının sistem kaynaklarının hangi uygulama tarafından kullanıldığını açık ve şeffaf biçimde görebilmesidir.

Foreground Service’in Zorunlu Olduğu Durumlar

Aşağıdaki görevler uzun süreli olduklarından foreground service gerektirir:

- Sürekli GPS konum takibi,
- VOIP arka plan servisleri (örn. WhatsApp araması),
- Müzik oynatma,
- Sağlık/fitness sensör takibi,

- Uzun süreli dosya yükleme/indirme,
- Arka planda kesintisiz mikrofon kaydı.

Bu işlerde kullanıcıya gösterilen bildirim, Android'in güvenlik ve pil politikaları gereği zorunludur.

Zamanlanmış Bildirimler Arka Plan Çalışma Sayılmaz

Zamanlanmış bildirimler (*scheduled notifications*), Android'in *AlarmManager* ve platform zamanlayıcıları aracılığıyla işletim sistemi tarafından tetiklenir. Bu tür bildirimlerin çalışması için uygulamanın:

- arka planda açık kalmasına,
- servis çalıştırmasına,
- foreground service başlatmasına,
- sürekli bir bildirim göstermesine,
- bir zamanlayıcı veya döngü (interval) tutmasına

gerek yoktur.

Uygulama tamamen kapalı olsa dahi işletim sistemi zaman geldiğinde bildirimi kendi başına tetikler.

Scheduled Notification Kullanım Örneği (Notiffee)

Aşağıdaki örnek, 5 saat sonrası için planlanmış bir ilaç hatırlatma bildiriminin Notiffee kütüphanesi ile nasıl oluşturulduğunu göstermektedir:

```
import notiffee, { TimestampTrigger, TriggerType }
  from '@notiffee/react-native';

const date = new Date(Date.now() + 5 * 60 * 60 * 1000);

const trigger: TimestampTrigger = {
  type: TriggerType.TIMESTAMP,
  timestamp: date.getTime(),
  alarmManager: true
};

await notiffee.createTriggerNotification(
  {
    title: 'İlaç Hatırlatması',
    body: 'İlaç saatiniz geldi.'
  },
  trigger
);
```

Bu bildirim, uygulama kapalı olsa bile işletim sistemi tarafından zamanında üretilecektir.

Neden Scheduled Notification İçin Foreground Service Gerekmez?

Çünkü scheduled notification bir *sürekli iş* değildir; yalnızca zaman geldiğinde bir kez çalışması gereken hafif bir tetikleyicidir. Android bu işi sistem seviyesinde güvenilir bir şekilde planlar. Dolayısıyla:

- Sürekli işlem yoktur.
- Pil tüketimi yoktur.
- Arka plan kodu uzun süre çalışmaz.
- Kullanıcıya sürekli bildirim göstermek gerekmez.

Sonuç

Android'de uzun süre çalışan görevler yalnızca foreground service aracılığıyla yürütülebilirken, zamanlanmış bildirimler doğrudan işletim sistemi tarafından tetiklenir ve herhangi bir arka plan işlemine ihtiyaç duymaz. Bu nedenle, ilaç hatırlatma gibi belirli bir zamanda çalışması gereken bildirimler için foreground service kullanılması gereksiz ve yanlıştır. Doğru yaklaşım, *scheduled notification* mekanizmasını kullanmaktır.

Android’de Zamanlanmış Bildirimlerde Gecikme Problemi

Android işletim sisteminde zamanlanmış bildirimlerin veya alarmların belirlenen saatten gecikmeli olarak tetiklenmesi sık görülen bir durumdur. Özellikle Notiffee gibi üçüncü taraf kütüphaneler aracılığıyla oluşturulan *Timestamp Trigger* tabanlı bildirimlerin yaklaşık 10–30 dakika öteye kayması, işletim sisteminin güç tasarrufu mekanizmalarından kaynaklanmaktadır. Bu bölüm, gecikmenin temel nedenlerini ve çözüm yöntemlerini açıklamaktadır.

Gecikmenin Temel Nedenleri

Zamanlanmış bir bildirimin bir gün sonra veya belirli aralıklarla yaklaşık 15 dakika ileri kayması genellikle aşağıdaki işletim sistemi davranışlarından kaynaklanır.

1. Doze Mode Sebebiyle Exact Alarmin Ertelenmesi

Android 6.0 ile tanıtılan *Doze Mode*, cihaz uzun süre hareketsiz kaldığında zamanlayıcıları, ağ erişimini ve arka plan yürütmesini sınırlayan bir güç tasarrufu mekanizmasıdır. Doze sırasında:

- Sadece *setExactAndAllowWhileIdle* tipi alarmlar çalıştırılır.
- Normal alarmlar *inexact* hale getirilerek uygun bir bakım penceresinde (maintenance window) tetiklenir.
- Bu bakım pencereleri genellikle 10–30 dakika aralıklarla gerçekleşir.

Bu nedenle exact olarak ayarlanamayan alarmlar yaklaşık 15 dakika gecikme ile tetiklenebilir.

2. İnexact Alarm Kullanımı

Bazı cihazlarda Notiffee veya AlarmManager, üretici (OEM) tarafından uygulanan güç optimizasyonu nedeniyle *setExact* komutunu *setInexact* biçimine dönüştürür. Bu durumda Android şu politikayı uygular:

“Belirtilen zaman yaklaşık bir hedeftir; alarm uygun sistem durumunda çalıştırılacaktır.”

İnexact alarmlar sistem tarafından tipik olarak 5–20 dakika ileriye kaydırılır.

3. Repeating (Tekrarlayan) Alarmlar Android Tarafından Kaydırılır

Android 6.0 sonrası:

- *setRepeating()* veya benzeri mekanizmalar exact değildir,

- Sistem bu alarmları enerji tasarrufu için düzenli biçimde geciktirir,
- Tipik kayma miktarı 10–15 dakikadır.

Bu nedenle günlük ilaç hatırlatmaları için tekrarlayan alarm kullanmak önerilmez.

4. OEM (Üretici) Pil Optimizasyonları

Bazı üreticiler (Samsung, Xiaomi, Oppo, Vivo) alarm davranışını ek güç koruma katmanları ile değiştirir. Bu işletim sistemi davranışları:

- exact alarmları inexact hale getirebilir,
- uygulamanın zamanlanmış tetikleyicilerini geciktirebilir,
- bakım pencerelerini daha seyrek hale getirebilir.

Beklenen gecikme süresi çoğu cihazda 10–30 dakika arasındadır.

Gecikmenin Çözüm Yöntemleri

Zamanlanmış bildirimlerdeki kaymayı azaltmak veya ortadan kaldırmak için aşağıdaki yöntemler kullanılmalıdır:

1. Exact Alarm Kullanımı

Notiffee üzerinde exact tetikleme için aşağıdaki yapılandırma önerilir:

```
const trigger = {
  type: TriggerType.TIMESTAMP,
  timestamp: targetTime,
  alarmManager: true,
  allowWhileIdle: true
};
```

Bu ayarlar Android'in `setExactAndAllowWhileIdle` çağrısını kullanmasını sağlayarak Doze Mode gecikmesini büyük ölçüde azaltır.

2. Tekrarlayan Alarm Kullanmaktan Kaçınma

Android, repeating alarmları inexact olarak işler. Bunun yerine:

- her tetikleme sonrasında bir sonraki alarmın zamanı manuel olarak yeniden hesaplanmalı,
- yeni bir exact alarm planlanmalıdır.

3. Pil Optimizasyonunu Devre Dışı Bırakma (İsteğe Bağlı)

Bazı OEM cihazlarında alarm davranışının tamamen doğru çalışması için kullanıcının:

- battery optimization (doze),
- background limits,
- autostart izinleri

gibi sistem kısıtlamalarını uygulama özelinde kapatması gerekebilir.

Sonuç

Android işletim sistemindeki güç tasarrufu politikaları nedeniyle zamanlanmış bildirimlerin exact biçimde tetiklenmesi garanti değildir. Doze Mode, inexact alarm dönüşümleri, tekrarlayan alarm kısıtlamaları ve OEM güç optimizasyonları, alarmların genellikle 10–15 dakika ileri kaymasına yol açar. Exact alarm kullanımı ve tekrarlayan alarmlardan kaçınılması, bu gecikmeleri en aza indiren temel stratejilerdir.

Ağ İşlemleri ve Sistem Çağrılarının Temel Mantığı

Mobil işletim sistemlerinde ağ iletişimi, doğrudan kullanıcı arayüzü iş parçacığını etkileyebilecek yüksek gecikmeli işlemler arasındadır. Bir ağ isteği, uygulama kodundan çekirdek seviyesindeki *system call* arabirimine, oradan modem sürücüsüne ve daha sonra mobil şebeke veya Wi-Fi donanımına iletilir. Bu katmanlı yapı gereği ağ işlemleri yüksek gecikme ve belirsizlik içerir.

Neden Ağ İşlemleri UI Thread'de Yapılmamalıdır?

Ağ isteğinin tamamlanması, bağlantı kalitesi, modem durumu, DNS çözümleme süresi, yavaş sunucu yanıtı veya paket kayıpları gibi dış faktörlere bağlıdır. Bu nedenle ağ bekleme süresi milisaniyeler değil, çoğu zaman yüzlerce milisaniye hatta saniyeler seviyesinde olabilir.

UI thread üzerinde ağ beklemesi yapılması:

- Çerçeve çizimlerini (frame rendering) durdurur,
- Dokunmatik olaylarının işlenmesini engeller,
- *ANR (Application Not Responding)* hatasına yol açabilir.

Örnek: Hatalı Kullanım

```
const res = await fetch("https://api.example.com/data");

// Bu kod UI olay döngüsünü bloke eder ve donmaya neden olabilir.
```

Örnek: Doğru Kullanım

```
// React Native örneği | asenkron çalışma
useEffect(() => {
  (async () => {
    const res = await fetch(...);
  })();
}, []);
```

Sonuç

Ağ işlemleri yüksek gecikmeli ve dış faktörlere bağlı olduğu için her zaman arka plan iş parçacığında veya asenkron mekanizmalarla yürütülmelidir. UI thread üzerinde kesinlikle yapılmamalıdır.

Ağ İsteklerinin İşlem Boru Hattı

Bir `fetch()` çağrısı veya `URLRequest` işlemi aşağıdaki katmanlardan geçer:

1. **Uygulama Seviyesi (User Space)** React Native, Kotlin, Swift veya Flutter kodu ağ isteğini oluşturur.
2. **Sistem Çağrılarını (System Calls)** Uygulama, `send()`, `connect()`, `recv()` gibi POSIX tabanlı çekirdek çağrılarını (syscall) tetikler.
3. **Kernel Networking Stack** Linux çekirdeği (Android) veya XNU çekirdeği (iOS) TCP/IP protokollerini, soket yönetimini, paket yönlendirmeyi, güvenlik politikalarını ve buffer yönetimini yürütür.
4. **Sürücüler (Drivers)** Kernel, modem veya Wi-Fi sürücüsüne talimat gönderir.
5. **Donanım (Modem / Wi-Fi / Radio)** Paket fiziksel olarak gönderilir veya alınır.

Bu zincirin her bir adımı gecikmeye açıktır ve geliştiricinin doğrudan kontrolünün dışındadır.

Ağ Gecikmesine Neden Olan Faktörler

Ağ isteğinin tamamlanma süresi sabit değildir. Aşağıdaki faktörler gecikmenin temel sebepleridir:

- **DNS çözümleme** Alan adının IP adresine çevrilmesi 5–150 ms arasında sürebilir.
- **TCP Üçlü El Sıkışması (Handshake)** SYN → SYN/ACK → ACK sırası 30–300 ms arası gecikme yaratabilir.
- **TLS kurulumu** HTTPS bağlantıları için ek 1–2 round-trip gerekir; mobil ağda gecikme ciddi ölçüde artabilir.
- **Paket kaybı ve yeniden iletim** Zayıf sinyalde paket kaybı ve gecikme katlanır.
- **Radio durumları (RRC State Machine)** Mobil cihazlarda modem genellikle düşük güç modundadır (Idle). Bir paket göndermek için modem önce:
 1. Idle → Connected geçişi yapar (50–200 ms),
 2. ardından ağ isteğini iletir.
- **Operatör gecikmesi** 4G/5G altyapısı, baz istasyonu yükü ve mobil çekim gücü gecikmeyi etkiler.

Bu nedenle ağ isteğinin toplam gecikmesi genellikle 100–200 ms civarında değil, çoğu zaman 300–800 ms seviyesindedir.

UI Thread Üzerinde Ağ İşlemlerinin Bloklanması

UI thread, çerçeve çizimi (frame rendering), animasyon yürütme, dokunma olaylarının işlenmesi ve bileşenlerin güncellenmesinden sorumludur. Bu thread'in aşağıdaki sebeplerle bloklanması ciddi sorunlar doğurur:

- UI güncellemelerinin gecikmesi,
- kullanıcının dokunma olaylarına yanıt verilememesi,
- animasyonların durması,
- uygulamanın donmuş gibi görünmesi,
- en sonunda *ANR (Application Not Responding)* hatası.

Özellikle mobil ağ gecikmesi değişken olduğundan UI thread'in bir ağ isteğini beklemesi kabul edilemez bir durumdur.

React Native Özelinde Ek Bir Risk: Bridge Bloklanması

React Native'de JavaScript tarafı ile native taraf arasındaki iletişim *bridge* üzerinden gerçekleşir. Eğer JS thread üzerinde ağır bir işlem yapılırsa:

- köprü üzerinden veri gidemez,
- UI thread güncellemeleri gecikir,
- animasyonlar durur,
- ekran yanıt vermez hale gelir.

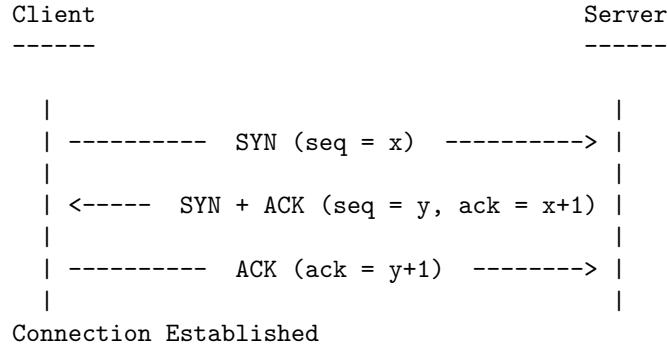
Bu durum özellikle büyük JSON'ların parse edilmesi, pahalı döngüler veya synchronization hataları nedeniyle sıkça yaşanır.

Ek Ağ ve İşletim Sistemi Kavramları (Kısa Teknik Özet)

Bu bölüm, mobil geliştiricilerin ağ gecikmeleri, modem davranışı ve iş parçacığı yönetimi konularında bilmesi gereken ileri düzey kavramları kısa fakat teknik biçimde özetlemektedir.

TCP/IP Üçlü El Sıkışması (Three-Way Handshake)

Bir TCP bağlantısı kurulmadan önce istemci ile sunucu arasında aşağıdaki sıralı paket alışverişi gerçekleşir:



Bu işlem en az bir tam RTT (Round Trip Time) gecikmesi gerektirir. Mobil ağlarda RTT yüksek olduğundan bağlantı kurulum gecikmesi UI performansını dolaylı olarak etkileyebilir.

Mobil Radyo Güç Durumları (RRC State Machine)

Mobil modemler enerji tasarrufu amacıyla farklı güç durumları arasında geçiş yapar. Temel iki durum:

- **Idle:** Modem düşük güç modundadır; veri aktarımı yoktur. Bu durumdan çıkıp bağlantı kurmak 50–200 ms gecikme yaratır.
- **Connected:** Modem tam güçte çalışır; veri aktarımı hızlıdır ancak pil tüketimi yüksektir.

RRC durum geçişleri, ağ isteğinin başlangıç gecikmesine katkıda bulunur.

TCP, IP, UDP, HTTP ve HTTPS Kavramlarının Temel Tanımları

Bu bölümde internet iletişimini oluşturan temel protokoller kısa ve teknik bir biçimde özetlenmektedir. Mobil geliştiriciler için ağ iletişiminin hangi katmanda nasıl gerçekleştiğini anlamak kritik öneme sahiptir.

IP (Internet Protocol)

IP, internet üzerinde paketlerin hangi hedefe gönderileceğini belirleyen adresleme protokolüdür. Temel işlevi yönlendirme (*routing*) ve hedefe ulaşım garantisiz ol-
madan paket iletimidir.

- Bağlantısızdır; gönderilen paketler kaybolabilir.
- Paketlerin sırası karışabilir.
- Yalnızca “adresleme” ve “iletim” görevini üstlenir.

TCP (Transmission Control Protocol)

TCP, IP'nin üzerinde çalışan güvenilir bir taşıma protokolüdür. Paketlerin doğru sırayla, eksiksiz ve güvenli şekilde karşı tarafa ulaşmasını sağlar.

- Bağlantı kurulumu için *Three-Way Handshake* kullanır.
- Paket kaybı durumunda yeniden iletim yapar.
- Veri akış kontrolü ve sıralama mekanizmaları içerir.
- API çağrılarının ve web isteklerinin temel taşıyıcısıdır.

UDP (User Datagram Protocol)

UDP, TCP'ye göre daha hızlı ancak güvensiz bir taşıma protokolüdür.

- Bağlantı kurulmaz; “en hızlı gönder ve geç” yaklaşımı.
- Paket kaybı veya sıra bozulması önlenmez.
- Oyunlar, canlı yayın, VoIP gibi düşük gecikme gerektiren alanlarda kullanılır.

HTTP (Hypertext Transfer Protocol)

HTTP, istemci ve sunucu arasındaki iletişimi tanımlayan üst seviye protokoldür. Web sayfaları, REST API çağrıları ve birçok mobil hizmet HTTP üzerinden çalışır.

- GET, POST, PUT, DELETE gibi metodlar içerir.
- Header, body, status code yapıları tanımlıdır.
- Çoğunlukla TCP üzerinde çalışır.
- Güvenlik sağlamaz; iletilen veri düz metin olabilir.

HTTPS (HTTP + TLS)

HTTPS, HTTP protokolünün TLS (Transport Layer Security) ile şifrelenmiş halidir.

- Veri şifrelenir (gizlilik).
- Veri bütünlüğü sağlanır (tamper protection).
- Sunucunun kimliği doğrulanır (authentication).
- Modern mobil uygulamalarda zorunlu olarak kullanılır.

Katmanlar Arası İlişki

Ağ isteği aşağıdaki katmanlardan geçerek iletilir:

Uygulama (fetch, axios, XMLHttpRequest)
↓
HTTPS (şifreleme + HTTP)
↓
HTTP (istemci{sunucu dili})
↓
TCP (güvenilir taşıma)
↓
IP (adresleme ve yönlendirme)
↓
Donanım (Wi-Fi / LTE / 5G modem)

Sonuç

IP adreslemeyi, TCP güvenilir iletimi, UDP düşük gecikmeli iletimi, HTTP uygulama seviyesindeki iletişimi, HTTPS ise güvenli veri aktarımını sağlar. Bu katmanların anlaşılması mobil ağ performansını optimize etmek için temel gerekliliktir.

DNS, TLS ve Paket Kaybının Etkisi

Ağ isteğinin ilk aşamalarında şu ek gecikmeler ortaya çıkar:

- **DNS çözümleme:** 5–150 ms.
- **TLS el sıkışması:** En az bir ek RTT gerektirir.
- **Paket kaybı:** Tek bir paket kaybı TCP yeniden iletim mekanizmasını tetikler ve gecikme katlanabilir.

Bu katmanlardaki gecikme arttıkça, UI thread üzerinde bloklama riski de aynı ölçüde artar.

React Native Thread Modeli

React Native, iş parçacıklarını üç veya dört farklı thread'e dağıtarak UI performansını korur:

- **UI Thread:** Bileşenlerin çizimi, dokunma olayları ve animasyonların işlenmesi.
- **JS Thread:** JavaScript kodunun çalıştığı ortam. Ağır JS işlemleri UI'ı dolaylı olarak bloke eder.
- **Shadow Thread:** Flexbox layout hesaplamaları bu thread üzerinde yapılır.
- **Fabric / JSI Thread (Yeni mimari):** UI güncellemeleri doğrudan C++ katmanında yürütülür; köprü (bridge) kullanımı azalır.

Bu thread'lerden birinin aşırı yüklenmesi jank'a yol açabilir.

Modern HTTP İstemcileri (Cronet, OkHttp, NSURLSession)

Mobil işletim sistemleri yüksek performanslı HTTP istemcileri kullanır:

- **Cronet (Chrome Networking):** Google'ın düşük gecikmeli, QUIC destekli HTTP motorudur. Birçok büyük uygulama (YouTube, Instagram) Cronet kullanır.
- **OkHttp (Android):** Android ekosisteminin varsayılan yüksek performanslı HTTP istemcisidir. HTTP/2 ve TLS optimizasyonları sunar.
- **NSURLSession (iOS):** Apple'ın sistem seviyesine gömülü ağ kütüphanesidir. Arka plan transfer desteği ve enerji optimizasyonlarıyla bilinir.

Bu istemciler ağ katmanındaki belirsizliği azaltmak ve performansı iyileştirmek üzere tasarlanmıştır.

İş Parçacıkları ve Eşzamanlılık (Threading ve Concurrency) Temelleri

Modern mobil işletim sistemleri (Android ve iOS), uygulama iş yükünü eşzamanlı olarak yürütebilmek için çok iş parçacıklı bir model kullanır. Kullanıcı arayüzü (UI) iş parçacığı çizim, animasyon ve dokunma olayları gibi zaman duyarlı görevlerden sorumludur. Bu nedenle ağır işlemlerin UI thread üzerinde çalıştırılması, uygulamanın donmasına, gecikmelere ve *ANR (Application Not Responding)* hatalarına yol açabilir.

İş Parçacığı Türleri

Mobil platformlarda yaygın olarak kullanılan thread türleri şöyledir:

- **UI Thread (Main Thread):** Çizim, layout hesaplamaları, animasyonlar ve dokunma olaylarının işlendiği ana iş parçacığıdır. Her kare (frame) yaklaşık 16.67 ms içinde işlenmelidir.
- **Background Threads:** Ağ istekleri, disk IO, JSON çözümleme veya yoğun hesaplama gibi zaman alan işlemler için ayrılmış iş parçacıklarıdır.
- **Scheduled / Worker Threads:** Zamanlanmış görevler, uzun süreli işler ve sistem tarafından yönetilen arka plan operasyonları için kullanılır.

Android'de bu yapı *Looper* ve *MessageQueue* üzerinden yönetilir; iOS'ta ise *Grand Central Dispatch (GCD)* ve *Dispatch Queues* kullanılır.

Neden Arka Plan Thread'leri Gereklidir?

Aşağıdaki işlemler yüksek gecikme (latency) veya yüksek CPU kullanımına sahip olduklarından UI thread üzerinde çalıştırılmaları sakıncalıdır:

- Ağ istekleri (TCP handshake, DNS, TLS kurulumu),
- Büyük dosya okuma/yazma işlemleri,
- Yoğun CPU gerektiren algoritmalar,
- Büyük JSON verilerinin parse edilmesi,
- Resim veya video işleme operasyonları.

Bu tür işlemler UI thread üzerinde yürütülürse çerçeve çizimleri zamanında tamamlanamaz ve kare düşmesi (*frame drop*) ortaya çıkar.

Race Condition ve Deadlock

Çok iş parçacıklı sistemlerde veri yarışları ve kilitlenme durumları kritik hata türleridir:

- **Race Condition:** Birden fazla thread aynı değişkene farklı zamanlarda erişip güncellediğinde veri tutarsızlığı oluşur.
- **Deadlock:** İki veya daha fazla thread birbirinin kilidini beklediğinde sistem tamamen kilitlenir.

Gerçek Örnek (Android):

- Thread A: kullanıcı bilgilerini okur ve günceller.
- Thread B: aynı kullanıcı bilgilerini diskten yeniden yükler.

Bu iki işlem uygun senkronizasyon olmadan yürürse, bellek durumu tutarsız hale gelebilir.

Örnek: Yanlış Kullanım

```
// React Native | ağır döngü JS thread'i tamamen bloke eder
for (let i = 0; i < 1_000_000_000; i++) {
  // hesaplama
}
```

Bu durumda JavaScript thread'i, UI thread ile iletişimi sağlayan köprüyü bloklar; animasyonlar, jestler ve dokunma olayları işlenemez hale gelir.

UI Thread'in Korunması İçin Öneriler

- Ağ ve disk işlemlerini daima background thread'e taşıyın.
- Ağır hesaplamaları ayrı worker thread'lerde yürütün.
- JS tarafında büyük JSON verilerini parse etmekten kaçın; mümkünse native tarafta işleyin.
- Gereksiz tekrar render oluşturan state değişikliklerini azaltın.
- Uzun süreli döngülerden kaçın; algoritmaları parçalayın.
- Animasyonlar sırasında layout değişikliklerini sınırlayın.

Sonuç

Eşzamanlılık, yüksek performanslı mobil uygulamaların belkemiğidir. UI thread üzerinde yürütülen yoğun işlemler hem kullanıcı deneyimini zayıflatır hem de ANR hatalarına yol açar. İş yükünün uygun şekilde arka plan thread'lerine dağıtılması, akıcı ve kararlı bir kullanıcı arayüzü için zorunludur.

Thread-Safe Olmayan İşlemlerin Belirlenmesi

Bir işlemin thread-safe olup olmadığını anlamak için şu kriterler göz önünde bulundurulur:

- **Paylaşılan Durum (Shared State):** Birden fazla thread tarafından erişilen değişkenler veya veri yapıları senkronizasyon olmadan güncelleniyorsa işlem thread-safe değildir.
- **Atomic Olmayan İşlemler:** Birden fazla adım içeren okuma-yazma operasyonları bölünemez (*atomic*) değilse race condition oluşabilir.
- **Global veya Statik Kaynak Kullanımı:** Statik koleksiyonlar, sayaçlar, cache yapıları veya singleton nesnelerinde eşzamansız erişim risklidir.
- **Yan Etki (Side Effect):** Metod, dış dünyada gözle görünür bir değişiklik yaratıyorsa (örneğin dosya yazma, veri tabanı güncelleme) thread-safe değildir veya uygun kilitleme gerektirir.
- **Dokümantasyon:** Android, iOS ve React Native API'ları çoğu zaman açıkça *thread-safe* veya *not thread-safe* olarak işaretlenmiştir.

Thread-safe olmayan işlemler uygun kilitleme (mutex), immutable veri yapıları veya tek thread kullanımıyla güvenli hale getirilebilir.

Animasyonların Neden UI Thread Üzerinde Çizildiği

Mobil işletim sistemlerinde animasyonlar kullanıcı arayüzü ile doğrudan ilişkili olduğu için UI thread üzerinde yürütülür. Bunun nedenleri:

- **Çizim Boru Hattı (Rendering Pipeline):** Görsel bileşenlerin ölçülmesi, yerleştirilmesi ve ekrana çizilmesi UI thread üzerinde gerçekleşir. Animasyon bu sürecin bir parçasıdır.
- **16.67 ms Kuralı:** 60 FPS akıcılığı sağlamak için her kare 16.67 ms içinde hesaplanmak zorundadır. UI thread bu süreyi kontrol eder.
- **Tutarlılık (Consistency):** UI bileşenlerinin tüm görsel değişimleri tek bir merkezi thread üzerinden işlenmezse tutarsız ve yırtılma (*tearing*) adı verilen hatalar ortaya çıkar.
- **Platform Politikası:** Android ve iOS, UI bileşenlerine arka plandan erişimi yasaklar; çünkü thread yarışları görsel bozulmaya yol açabilir.

Bu nedenle animasyon hesaplamaları UI dışındaki thread'lerde yapılabilir ancak **çizim güncellemeleri mutlaka UI thread'de uygulanır.**

JavaScript Thread Optimizasyon Teknikleri (React Native)

React Native’de JavaScript thread’in aşırı yüklenmesi, köprü iletişiminin gecikmesine ve UI thread’in dolaylı olarak bloklanmasına sebep olur. JS thread’i optimize etmek için aşağıdaki teknikler uygulanmalıdır:

- **Ağır Hesaplamaları Parçalama:** Büyük döngüler veya karmaşık algoritmalar *chunk*lara bölünmeli; her parça `setTimeout` veya `InteractionManager` ile zamanlanarak yürütülmelidir.
- **Büyük JSON Veri Setlerini Native Tarafa İşleme:** JSON parse işlemleri pahalıdır; büyük veriler için native modüller veya JSI tabanlı çözümler kullanılmalıdır.
- **Gereksiz Renderları Azaltma:** `React.memo`, `useCallback`, `useMemo` ve key tabanlı optimizasyonlar ile yeniden render sayısı azaltılmalıdır.
- **Animasyonların Native Tarafa Taşınması:** `react-native-reanimated` veya `LayoutAnimation` kullanarak animasyon hesaplamalarını native thread’de yürütmek JS thread yükünü ciddi ölçüde düşürür.
- **Bridge Üzerinden Büyük Veri Transferinden Kaçınma:** JS-Native köprüsünden büyük objeler göndermek darboğaz yaratır. Bunun yerine JSI veya turbo module mimarisi kullanılmalıdır.
- **Async Storage, File IO ve Crypto İşlemlerini JS’de Yapmama:** Bu işlemler native thread’lerde çalıştırılmalı, JS üzerindeki bloklama engellenmelidir.

Bu teknikler uygulandığında JS thread daha akıcı çalışır ve UI performansında gözle görülür bir iyileşme elde edilir.

Sandboxing ve Uygulama İzolasyonu

Mobil işletim sistemleri (Android ve iOS), güvenlik, gizlilik ve sistem kararlılığını sağlamak amacıyla uygulamaları birbirinden kesin sınırlarla ayırarak çalıştırır. Bu mekanizma *sandboxing* olarak adlandırılır. Her uygulama, kendisine ait dosya sistemi alanı, süreç kimliği ve izin kümesi ile sınırlandırılmıştır. Bu yapı, uygulamaların diğer uygulamalara veya sisteme zarar vermesini engelleyen temel güvenlik katmanıdır.

Sandbox Mekanizmasının Hedefleri

Sandboxing'in amacı yalnızca uygulamaları birbiriyle izole etmek değildir. Aynı zamanda işletim sistemi düzeyinde kapsamlı güvenlik sağlar:

- **Veri Güvenliği:** Uygulamalar yalnızca kendi veri klasörlerine erişebilir ve başka uygulamaların dosya sistemine erişemez.
- **Bellek İzolasyonu:** Bir uygulama, başka bir sürecin RAM'de tuttuğu verilere erişemez. Bellek adres alanları tamamen ayrılmıştır.
- **Yetki Sınırlandırma:** Her uygulama yalnızca kendisine verilen izinler dahilinde işlem yapabilir (kamera, mikrofon, konum vb.).
- **Hata İzolasyonu:** Bir uygulamanın çökmesi diğer uygulamaların veya işletim sisteminin çalışmasını etkilemez.
- **Zararlı Yazılım Koruması:** Uygulamalar birbirlerine müdahale edemediği için kötü niyetli yazılımların diğer uygulamaların verilerini okuması engellenir.

Süreç Seviyesinde İzolasyon

Android ve iOS, sandboxing'in en temel ayağını süreç (process) seviyesinde uygular.

- **Android:** Her uygulama kendi benzersiz kullanıcı kimliği (UID) altında çalışır. Bu UID, Linux çekirdeği tarafından uygulamanın dosya sistemine erişim haklarını belirlemede kullanılır.
- **iOS:** Uygulamalar, “application container” adı verilen özel bir dizinde çalışır ve süreçleri App Sandbox profili ile sınırlandırılır. Ek yetkiler *entitlement* dosyalarıyla belirlenir.

Süreç kimliği ayrımı sayesinde bir uygulama başka bir uygulamanın dosyalarına veya belleğine erişemez.

Dosya Sistemi Sınırları

Uygulamanın erişebildiği dosya sistemi alanı açık şekilde sınırlandırılmıştır:

- **Android:** Her uygulama `/data/data/<package>` dizininde kendi özel alanına sahiptir. Diğer dizinlere erişim çekirdek seviyesinde engellenir.
- **iOS:** Uygulama konteynerleri `Documents`, `Library` ve `tmp` klasörlerini içerir. Kapsayıcı dışındaki dosyalara erişim mümkün değildir.

Bu yapının sonucu olarak:

- Başka bir uygulamanın SQLite veritabanına erişilemez.
- Başka bir uygulamanın cache dosyaları okunamaz.
- Uygulamanın kendisine ait olmayan dizinlere yazması yasaktır.

İzinler ve Yetkilendirme (Permissions ve Entitlements)

Uygulamalar yalnızca işletim sisteminin kendilerine açıkça izin verdiği kaynaklara erişebilir.

- **Android:** Kamera, mikrofon, konum, sensörler gibi donanımlar *runtime permission* mekanizmasıyla denetlenir. Uygulamanın izin almadan bu kaynaklara erişmesi engellenir.
- **iOS:** Uygulamalar belirli yetkileri Info.plist ve entitlements üzerinden talep eder. Kullanıcı bu yetkileri açıkça onaylamadıkça erişim verilmez.

Bu model, sandbox'ın yalnızca dosya sistemi değil donanım seviyesinde de koruma sağladığını gösterir.

IPC (Inter-Process Communication) Kısıtlamaları

Sandboxing, uygulamalar arasında serbest veri paylaşımını yasaklar. İşletim sistemleri yalnızca sınırlı IPC mekanizmaları sunar:

- Android: Binder, PendingIntent, ContentProvider
- iOS: App Groups, URL Scheme, UIPasteboard

Bu mekanizmalar kontrollü ve güvenli veri paylaşımı sağlar; keyfi erişime izin verilmez.

React Native Üzerindeki Etkileri

React Native uygulamaları da aynı sandbox kurallarına tabidir:

- JS tarafı, yalnızca native modüllerin izin verdiği kaynaklara erişebilir.
- Uygulama sandbox dışındaki klasörleri okuyamaz.
- Bridge veya JSI üzerinden bile başka uygulamaların dosyalarına erişim mümkün değildir.
- Resim, video, dosya seçimi ancak kullanıcı müdahalesiyle (file picker) yapılabilir.

Örnek Senaryolar

- x uygulamasının y'nin medya klasörüne erişmeye çalışması sandbox tarafından engellenir.
- Android'de başka bir uygulamaya ait “shared preferences” dosyasını okumak imkânsızdır.
- iOS'ta bir uygulamanın diğer uygulamanın fotoğraflarına erişmesi yalnızca kullanıcı manuel seçim yaptığıında mümkündür.

Sonuç

Sandboxing, mobil işletim sistemlerinin güvenlik mimarisinin temel bileşenidir. Her uygulamanın yalnızca kendi konteyneri içinde çalışması, veri sızıntılarını, kötü niyetli müdahaleleri ve süreçler arası bellek erişimini tamamen engeller. Bu sayede mobil cihazlar, çok uygulamalı ortamlarda dahi yüksek güvenlik ve kararlılık sağlar.

Process ve Lifecycle Yönetimi (Android ve iOS Karşılaştırmalı)

Mobil işletim sistemlerinde uygulamaların yaşam döngüsü ve süreç (process) yönetimi, hem kaynak kullanımını optimize etmek hem de kullanıcı deneyimini korumak amacıyla sıkı bir şekilde denetlenir. Android ve iOS, process yönetimi konusunda farklı mimarilere sahip olmakla birlikte her iki sistem de uygulamaların bellek, CPU ve enerji tüketimini düzenlemek için benzer yaklaşımlar kullanır.

Android’de Process ve Activity Lifecycle

Android uygulamaları, bir veya birden fazla *activity*, *service* ve *broadcast receiver* bileşeninden oluşur. Activity lifecycle, kullanıcı arayüzü deneyimini yönetmek için tanımlanmış bir durum makinesidir.

Activity Lifecycle Durumları

- **onCreate:** Activity oluşturulur; UI bileşenleri yüklenir.
- **onStart:** Activity görünür hale gelir ancak etkileşim henüz başlamamıştır.
- **onResume:** Activity kullanıcı ile etkileşimde; UI tamamen “foreground” dadır.
- **onPause:** Activity kısmen görünürdür; bazı işlemler durdurulabilir.
- **onStop:** Activity artık görünmez; arka plan görevleri minimal tutulur.
- **onDestroy:** Activity tamamen yok edilir.

Bu lifecycle, uygulamanın kullanıcıya nasıl sunulduğunu belirler; ancak uygulamanın süreç önceliği ve OS tarafından ne zaman öldürüleceği farklı bir mekanizmaya bağlıdır.

Android Process Priority (Öncelik Sıralaması)

Android, uygulama süreçlerini Linux çekirdeği üzerinde çalıştırır ve her süreç belirli bir öncelik sınıfına atanır. Bu sınıflar, sistemin bellek kritik seviyeye ulaştığında hangi uygulamaları kapatacağını belirler.

- **Foreground Process:** Kullanıcı aktif olarak bu süreç ile etkileşimdedir. En yüksek önceliğe sahiptir; kolay kolay öldürülmez.
- **Visible Process:** Activity görünür durumdadır ancak kullanıcı etkileşimde olmayabilir.
- **Service Process:** Arka planda çalışan servislerdir; orta seviyede önceliğe sahiptir.

- **Background Process:** Activity görünür değildir; yalnızca durumu RAM’de tutulur.
- **Cached Process:** Sistem performansı için saklanan ancak gerektiğinde ilk öldürülecek süreçlerdir.

Bu sistem sayesinde Android, düşük RAM koşullarında agresif biçimde arka plandaki süreçleri sonlandırabilir.

Low Memory Killer ve OOM (Out of Memory) Mekanizması

Android, Linux çekirdeğine entegre edilmiş bir *Low Memory Killer (LMK)* mekanizması kullanır. Sistem bellek baskısı altında olduğunda LMK, öncelik sınıflarına göre süreçleri sırayla öldürür.

- Foreground process *en son*,
- Cached ve background process *ilk önce* öldürülür.

Her cihazın bir OOM eşiği (threshold) bulunmaktadır. Uygulama, cihaz RAM limitine yakın bir bellek kullanımına ulaştığında OOM sinyaliyle zorla kapatılabilir.

Arka Planda Uygulamanın Öldürülme Nedenleri

- Yetersiz cihaz belleği,
- Diğer uygulamaların foreground’a geçmesi,
- OS’in agresif güç yönetimi,
- Doze Mode ve App Standby sınırlamaları,
- OEM optimizasyonları (özellikle Xiaomi, Oppo, Vivo).

Bu nedenle Android’deki “application killed in background” durumları sıkça görülür ve normaldir.

iOS'ta Uygulama Durumları (App States)

iOS, uygulamaların yaşam döngüsünü daha sade ancak daha katı şekilde yönetir. Uygulamalar aşağıdaki beş durumda bulunabilir:

- **Not Running:** Uygulama çalışmıyor veya OS tarafından sonlandırılmıştır.
- **Inactive:** Uygulama ekrandadır ancak etkileşim alamaz (örn. bildirim geldiğinde ekran kararma anı).
- **Active:** Uygulama tamamen kullanım halindedir.
- **Background:** Uygulama kısıtlı süreyle arka planda çalışabilir (genellikle 3–10 saniye).
- **Suspended:** Uygulama RAM'de tutulur fakat hiçbir kod çalışmaz. OS bellek ihtiyacı duyarsa bu uygulamayı öldürür.

Android'den farklı olarak iOS, “background execution” konusunda çok daha katıdır ve uygulamalar çoğu zaman *suspended* durumunda bekler.

iOS Background Execution Limits

iOS, uygulamanın arka planda çalışmasına yalnızca belli koşullarda izin verir:

- Aktif ses oynatma,
- Harita navigasyonu (GPS),
- VoIP aramaları,
- Bluetooth aksesuar iletişimi,
- Background fetch (işletim sistemi tarafından sınırlı zaman aralığında),
- Uzun süreli yükleme/indirme (NSURLSession background tasks).

Bu kategorilerin dışında kalan işlemler birkaç saniye içinde durdurulur ve uygulama *suspended* durumuna alınır.

iOS Memory Warning Mekanizması

iOS, bellek baskısı oluştuğunda uygulamalara *memory warning* iletir. Geliştiricilerin bu uyarıya yanıt olarak:

- büyük cache'leri temizlemesi,
- kullanılmayan nesneleri serbest bırakması,
- UI'da yer kaplayan büyük objeleri boşaltması

beklenir. Uygulama memory warning'e rağmen çok fazla bellek kullanmaya devam ederse OS tarafından sonlandırılır.

Android ve iOS Arasındaki Temel Farklar

- Android çok bileşenli (activity, service) bir model kullanır, iOS ise tek “application state” modeli kullanır.
- Android, arka plan uygulamalarını daha agresif şekilde öldürür, iOS ise uygulamayı öncelikle suspended durumuna alır.
- iOS, background execution konusunda Android’e göre çok daha kısıtlayıcıdır.
- Android LMK donanım seviyesinde çalışır, iOS memory pressure notification ile daha yumuşak bir yaklaşım sunar.

Sonuç

Android ve iOS, uygulamaların yaşam döngüsünü yönetirken farklı yaklaşımlar benimser. Android süreç önceliği ve LMK davranışıyla agresif bir model izlerken iOS daha sıkı arka plan kısıtlamalarına sahiptir. Her iki platformda da lifecycle yönetimini anlamak, arka plan işlemleri, bildirimler, performans optimizasyonu ve uygulama kararlılığı için kritik öneme sahiptir.

Bellek Yönetimi: GC, ARC, Leak Örüntüleri ve Memory Ownership

Mobil uygulamalarda bellek yönetimi, performans, kararlılık ve enerji tüketimi açısından kritik öneme sahiptir. Android ve iOS bu konuda farklı yaklaşımlar kullanır: Android, *ART Garbage Collector* ile otomatik bellek yönetimi sunarken; iOS, *Automatic Reference Counting (ARC)* mekanizmasını kullanır. React Native ise her iki platformun üstünde çalışır ve ayrıca JavaScript çalışma zamanının (JS engine) kendi heap yönetimine sahiptir. Bu bölümde her üç ortamın bellek modeli ayrıntılı olarak ele alınmaktadır.

Android ART Garbage Collector

Android RunTime (ART), bellek yönetimi için generational (kuşaksal) yaklaşımli bir garbage collector (GC) kullanır. ART GC'nin temel özellikleri şunlardır:

- **Generational Heap:** Bellek *young*, *old* ve *large object space* bölgelerine ayrılır.
- **Copying GC:** Young space üzerindeki objeler hızlıca kopyalanır; kısa ömürlü nesneler burada toplanır.
- **Mark-and-Sweep:** Old space için işaretleme ve temizleme yaklaşımı kullanılır.
- **Concurrent Collector:** GC çoğunlukla arka planda eşzamanlı çalışır.

GC genel olarak görünmezdir; ancak belirli durumlarda “stop-the-world” (uygulamanın tamamen durması) fazları yaşanabilir.

Stop-the-World Fazları

GC her zaman tam olarak eşzamanlı çalışmaz. Bazı aşamalarda tüm iş parçacıkları durdurulur. Bu fazlara *stop-the-world* denir.

- Root set taraması,
- Objelerin mark safhası,
- Finalize edilecek nesnelerin işaretlenmesi,
- Heap compaction (sıkıştırma).

Bu fazlar nanosaniye–milisaniye arası sürebilir; ancak büyük heaplerde, özellikle düşük bellekli cihazlarda gözle görülür jank oluşabilir.

Android’de Memory Leak Örüntüleri

Android uygulamalarında sık görülen leak türleri şunlardır:

- **Activity Leak:** Uzun ömürlü singleton veya static referansların activity nesnesini tutması.
- **Anonymous Inner Class Leak:** İç sınıfların dış sınıfa (örn. Activity) implicit referans taşıması.
- **Listener / Callback Unregister Edilmemesi:** BroadcastReceiver, LocationCallback gibi yapılar unregister edilmezse leak oluşur.
- **Large Bitmap Leak:** Büyük görsellerin heap üzerinde tutulması.
- **Context Leak:** Uygun olmayan yerde Activity context kullanılması.

Bu leak’ler GC tarafından otomatik temizlenemez çünkü nesneler referans grafında ulaşılabilir durumdadır.

iOS Automatic Reference Counting (ARC)

iOS bellek yönetimini referans sayma (reference counting) yöntemiyle gerçekleştirir. ARC, her nesneyi:

- oluşturulduğunda retain eder,
- kullanılmadığında release eder,
- referans sayısı sıfıra indiğinde bellekten siler.

Bu modelde GC yoktur; bellek tahliyesi deterministiktir. Ancak yanlış kullanımda *retain cycle* oluşabilir.

iOS Retain Cycle Problemi

Retain cycle, iki nesnenin birbirine güçlü referans (strong reference) tutması nedeniyle referans sayılarının asla sıfırlanmamasıdır.

Örnek tipik senaryo:

- Bir ViewController, bir closure'ı güçlü biçimde retain eder.
- Closure, ViewController'ı `self` ile retain eder.

Bu durumda GC olmadığı için memory leak kalıcı hale gelir. Retain cycle çözümü:

- Weak veya unowned referans kullanımı,
- Objelerin yaşam döngüsünün manuel düzenlenmesi.

React Native ve JavaScript Heap Yönetimi

React Native çalıştığında JavaScript motoru (Hermes veya JSC) bağımsız bir heap alanı yönetir. JS tarafında GC yapılır ancak bu GC:

- native heap'ten tamamen bağımsızdır,
- bridge üzerinden büyük veri geçişlerinde gecikmeye yol açabilir,
- senkron (stop-the-world) fazları daha belirgin olabilir.

RN'de tipik leak örüntüleri:

- Uzun ömürlü closure içinde büyük state saklamak,
- Event listener'ları temizlememek,
- Büyük JSON objelerini global scope'ta tutmak,
- Navigation parametrelerinde büyük nesne taşımak.

Bu leak'ler JS heap'i büyütür ve GC gecikmelerine neden olur.

Native Modüllerde Memory Ownership

React Native'de native modüller aracılığıyla Java veya Objective-C / Swift tarafında bellek yönetimi yapılır. Burada sorumluluk tamamen geliştiricidedir.

- **Java tarafı (Android):** ART GC yönetir ancak JNI ile yapılan native kod çağrılarında bellek elle yönetilmelidir.
- **iOS tarafı (Objective-C / Swift):** ARC çoğu durumu yönetir; ancak C/C++ objeleri ve Core Foundation türleri için manuel `CFRelease()` gerekebilir.
- **JSI Modülleri:** C++ tarafında bellek yönetimi tamamen manuel veya akıllı pointer (`std::shared_ptr`) ile yapılmalıdır.

Yanlış ownership modeli:

- native $\rightarrow$ JS referansı bırakılmazsa,
- JS $\rightarrow$ native objesi tutulmaya devam ederse,
- circular reference oluşursa,

hem JS heap'te hem native heap'te leak oluşabilir.

Sonuç

Bellek yönetimi mobil uygulama geliştirme sürecinin temel parçalarından biridir. Android, ART GC ile otomatik bellek temizleme sağlarken iOS, ARC modeli ile deterministik bir yaklaşım sunar. React Native ise bu iki sistemin üstünde çalışan bağımsız bir JavaScript heap ortamına sahiptir. GC durakları, retain cycle'lar, context leak'leri ve ownership hataları performans, pil tüketimi ve uygulama kararlılığı üzerinde doğrudan etkiye sahiptir.

İzin Sistemi Derinlemesine İnceleme (Android ve iOS)

Mobil işletim sistemleri, kullanıcı gizliliğini korumak amacıyla kamera, mikrofon, konum, dosya erişimi ve sensör kullanımını sıkı izin mekanizmaları ile denetler. Android ve iOS izin sistemleri birbirinden farklı çalışsa da temel amaç aynıdır: kullanıcıyı bilgilendirmek ve kötü niyetli erişimi engellemek. Bu bölümde iki platformun izin modelleri karşılaştırmalı ve detaylı şekilde açıklanmaktadır.

Android Permission Modeli

Android izinleri iki ana kategoriye ayrılır:

- **Normal Permissions:** Kullanıcıya risk oluşturmaz. Örn: internet, bluetooth, vibrate. Kullanıcıya diyalog gösterilmez; otomatik verilir.
- **Dangerous Permissions:** Kullanıcı gizliliği için risklidir. Runtime'da izin istemek zorunludur. Örn: kamera, mikrofon, konum, rehber, SMS, dosya erişimi.

Tehlikeli izinler *permission group* yapısına bağlıdır. Örneğin:

- CAMERA → Camera group
- READ_CONTACTS → Contacts group
- ACCESS_FINE_LOCATION → Location group

Bir izin grubu verildiğinde çoğu zaman aynı gruptaki diğer izinler sessizce verilebilir.

Android Runtime Permission Akışı

Android'de *dangerous permissions* (tehlikeli izinler) için izlenmesi gereken standart çalışma akışı aşağıdaki gibidir:

1. İzin manifest'de tanımlanır.
2. Uygulama çalışma zamanında izin ister.
3. Kullanıcı izin verir, reddeder veya "*Never Ask Again*" seçer.
4. OS sonucu uygulamaya döner.

Android 12+ sürümlerinde kamera/mikrofon erişimlerinde OS durum çubuğunda yeşil gösterge bulunur. Bu, kullanıcı şeffaflığı için zorunlu bir güvenlik mekanizmasıdır.

Android 11+ Permission Revocation

Android 11 ile birlikte sistem kullanılmayan uygulamaların izinlerini otomatik olarak geri alır (*auto-reset*).

- Uygulama uzun süre açılmazsa (yaklaşık 3 ay),
- OS tehlikeli izinleri otomatik sıfırlar,
- Kamera, konum, mikrofon yeniden sorulmak zorundadır.

Bu durum özellikle uzun süre kullanılmayan sağlık veya alarm uygulamalarında izin hatalarına yol açabilir.

Foreground ve Background Location İzinleri

Konum izinleri Android'de en karmaşık izin türüdür. Üç seviyede yönetilir:

- **ACCESS_COARSE_LOCATION:** Yaklaşık konum (Wi-Fi, BTS).
- **ACCESS_FINE_LOCATION:** Hassas konum (GPS).
- **ACCESS_BACKGROUND_LOCATION:** Uygulama kapalıyken konum izleme. Kullanıcıya tamamen ayrı bir ekran ile sorulmalıdır.

OS, arka plan konumu kötüye kullanımı engellemek için:

- Kullanıcıya özel açıklama ekranı sunar,
- Arka plan konumu için ek bir dialog tetikler,
- Play Store politikaları gereği ekran görüntüsü ister,
- Uygulamayı fazla sık kullanmıyorsa tarayıp devre dışı bırakabilir.

Bu nedenle background location, geliştiricilerin en çok hata yaptığı izin türlerinden biridir.

iOS Permission Modeli

iOS, izinleri Android'e kıyasla daha sıkı yönetir. Her izin için *NS*UsageDescription* anahtarının Info.plist içinde açıklanması zorunludur.

Temel iOS izinleri:

- Kamera — *NSCameraUsageDescription*
- Mikrofon — *NSMicrophoneUsageDescription*
- Konum — *NSLocationWhenInUseUsageDescription*, *NSLocationAlwaysAndWhenInUseUsageDescription*
- Fotoğraf galerisi — *NSPhotoLibraryUsageDescription*
- Sağlık verileri — HealthKit izinleri

iOS, Android'den farklı olarak izin grubu kavramı kullanmaz. Her izin ayrı ayrı yönetilir ve her biri için kullanıcıya açıklama zorunludur.

iOS Entitlements

İzin modeli dışında iOS'ta *entitlement* adı verilen yetkilendirme dosyaları bulunur. Bunlar uygulamanın sistem seviyesinde hangi kaynaklara erişebileceğini belirler.

Örnek entitlements:

- **com.apple.developer.healthkit** HealthKit erişimi.
- **com.apple.developer.networking.wifi-info** Wi-Fi bilgisi okuma.
- **com.apple.developer.pushkit.unrestricted-voip** VoIP arka plan modu.
- **com.apple.developer.siri** Siri entegrasyonu.
- **com.apple.security.application-groups** Uygulamalar arası ortak dizin paylaşımı.

Yanlış tanımlanan bir entitlement uygulamanın App Store'dan reddedilmesine neden olabilir.

iOS Kamera ve Mikrofon Güvenliği

iOS, kamera ve mikrofon kullanımını donanım seviyesinde sınırlar:

- Uygulama arka planda iken kamera kullanamaz.
- Mikrofon yalnızca belirli background modlarında aktif kalabilir (örn. VoIP, audio playback).
- Kamera kullanımı sırasında başka bir uygulama kameraya erişemez.
- Kullanıcı ekranın sağ üst köşesinde yeşil (kamera) veya turuncu (mikrofon) gösterge görür.

Bu sınırlamalar uygulama gizlice kamera/mikrofon açmasını diye tasarlanmıştır.

iOS Konum İzinleri: When-in-Use ve Always

iOS konum izinleri iki seviyede tanımlanır:

- **When In Use:** Uygulama ekrandayken konum erişimi.
- **Always:** Arka planda konum erişimi. Kullanıcıya iki aşamalı dialog sunulur.

iOS, Android'den daha katı şekilde:

- Always iznine geçişi açıkça kullanıcıya gösterir,
- Kullanıcının izni ayarlar üzerinden daha sonra düşürmesine izin verir,
- Uygulamanın konumu ne sıklıkta kullandığını raporlar ("X app used your location in the background").

Arka plan konumu hem App Store politikaları hem de OS kısıtlamaları açısından en hassas izinlerden biridir.

Sonuç

Android ve iOS izin sistemleri, kullanıcı gizliliğini korumak için farklı yaklaşımlar benimser. Android, izinleri tehlike seviyesine göre gruplayarak yönetirken iOS her izin için ayrı açıklama ve kontrol mekanizmaları sunar. Konum, kamera ve mikrofon gibi hassas izinlerde iki platform da şeffaflık, güvenlik ve kullanıcı kontrolünü önceliklendirmektedir. Geliştiricilerin izin akışlarını doğru tasarlaması uygulama kararlılığı, App Store uyumu ve güvenilirlik açısından kritik öneme sahiptir.

Depolama Sistemi: Scoped Storage, MediaStore ve iOS Containers

Mobil işletim sistemlerinde dosya erişimi, güvenlik ve gizlilik nedeniyle sıkı şekilde sınırlandırılmıştır. Android ve iOS, uygulamaların yalnızca kendi iznili alanlarında veri okumasına ve yazmasına izin verir. Bu bölüm, Android 10 ile tanıtılan *Scoped Storage*, MediaStore mimarisi, iOS dosya konteynerleri, File Provider yapısı ve App Groups gibi gelişmiş konuları ayrıntılı biçimde ele almaktadır.

Android Scoped Storage

Android 10 (API 29) ile birlikte, uygulamaların harici depolamaya erişimi tamamen yeniden tasarlanmıştır. Bu modele **Scoped Storage** adı verilir. Temel hedefler:

- Kullanıcı gizliliğini korumak,
- Uygulamaların birbirlerinin medya dosyalarını görmesini engellemek,
- Dosya erişimini izin tabanlı hale getirmek,
- Harici depolamada veri karmaşasını azaltmak.

Scoped Storage altında:

- Uygulamalar yalnızca **kendi özel klasörlerine** tam erişim sağlar (`/Android/data/<package>`),
- Ortak medya dosyalarına ancak MediaStore aracılığıyla erişilebilir,
- Doğrudan dosya yolu (file path) erişimi büyük ölçüde yasaktır.

Bu modelle birlikte kullanıcı fotoğrafları, videoları ve multimedya dosyaları yalnızca arayüz üzerinden veya MediaStore API aracılığıyla erişilebilir hale gelmiştir.

MediaStore API

MediaStore, Android'in ortak medya içeriklerine erişmek için sunduğu içerik tabanlı (*ContentProvider*) arayüzdür. MediaStore ile:

- Fotoğraflar,
- Videolar,
- Ses dosyaları,
- İndirilenler (Downloads)

içerik URI'leri üzerinden yönetilir. Temel avantajları:

- Uygulama dosya sistemine doğrudan erişmeden medya bulabilir,
- Silme ve düzenleme işlemleri için OS kullanıcıdan onay ister,
- Dosya yolu yerine güvenli URI kullanılır.

Örneğin bir fotoğrafı okumak için:

```
content://media/external/images/media/<id>
```

Klasik `/storage/emulated/0/DCIM/...` erişimi artık çoğunlukla kısıtlanmıştır.

Storage Access Framework (SAF)

Scoped Storage'ın izin verdiklerinden daha geniş erişim gerektiğinde kullanıcıya dosya veya klasör seçme ekranı gösterilir. Bu mekanizma *Storage Access Framework (SAF)* olarak adlandırılır.

SAF şunları sağlar:

- Uygulama doğrudan tüm depolamayı göremez,
- Kullanıcı bir dosya veya klasör seçerek uygulamaya erişim verebilir,
- OS her erişim için Content URI üretir,
- Uygulama yalnızca seçilen URI'ler üzerinde işlem yapabilir.

SAF tipik olarak PDF okuyucular, dosya yöneticileri ve bulut uygulamaları tarafından kullanılır.

iOS Depolama Modeli: Sandbox Containers

iOS, uygulamaları tamamen izole edilmiş **sandbox container** yapılarında çalıştırır. Her uygulamanın kendi kök dizini:

`/var/mobile/Containers/Data/Application/<UUID>/`

içerisinde bulunur. Bu konteyner altındaki klasörler:

- **Documents:** Kullanıcı tarafından görülebilen kalıcı dosyalar.
- **Library:** Ayarlar, cache alt dizini ve uygulamaya özgü veriler.
- **Library/Caches:** Silinebilir, yeniden oluşturulabilir dosyalar.
- **tmp:** Geçici dosyalar; OS tarafından istenildiğinde silinebilir.

iOS'ta uygulamalar birbirlerinin izinlerine erişemez; sandbox sınırları Android'den bile daha katıdır.

iOS File Providers

iOS, üçüncü parti depolama servisleri (Google Drive, Dropbox, OneDrive) ile entegrasyon için *File Provider* uzantıları kullanır.

Bu sistem:

- Uygulamaya “ziyaretçi dosyaları” (external files) sunar,
- Tüm erişimi Content-like bir API ile yönetir,
- Dosyalar uygulamanın sandbox'ına kopyalanmadan kullanılabilir,
- iPadOS dosya sisteminde gerçek bir “mini dosya yöneticisi” deneyimi sağlar.

iOS App Groups ile Veri Paylaşımı

Aynı geliştiriciye ait birden fazla uygulama veya bir uygulama ile widget / extension arasında veri paylaşımı gerektiğinde *App Groups* kullanılır.

App Groups, iki farklı uygulamaya ortak bir konteyner sağlar:

`/Library/GroupContainers/<group-id>/`

Bu mekanizma özellikle:

- Widget verilerinin ana uygulamayla paylaşılması,
- iMessage extension verileri,
- Siri extension entegrasyonu,
- Paylaşılan SQLite veya UserDefaults

için kullanılır.

Geçici ve Kalıcı Depolama Davranışları

Hem Android hem iOS, geçici (temporary) ve kalıcı (persistent) saklama alanlarını farklı şekilde değerlendirir.

Android

- **cache/** dizini OS tarafından temizlenebilir,
- **files/** kalıcıdır,
- **external cache** büyük yüklemeler için uygundur,
- **external files** yalnızca uygulama tarafından erişilebilir.

iOS

- **tmp/** dizini cihaz yeniden başlatıldığında veya depolama azalınca silinebilir,
- **Library/Caches** OS tarafından temizlenebilir,
- **Documents** kullanıcı tarafından görülebilir ve kalıcıdır,
- **App Groups** paylaşılan kalıcı alan sağlar.

Sonuç

Depolama sistemi, modern mobil işletim sistemlerinde güvenlik ve gizlilik açısından merkezi bir rol oynar. Android Scoped Storage modeli, harici depolamayı URI tabanlı erişime dönüştürürken, iOS sandboxing ile tam izolasyon sağlar. MediaStore, SAF, File Providers ve App Groups gibi mekanizmalar geliştiricilere güvenli ve kontrollü dosya yönetimi seçenekleri sunar. Depolama modelini doğru anlamak, veri kaybı, izin hataları ve yanlış dosya erişimi gibi yaygın sorunları önlemek için kritiktir.

İşletim Sistemi Zamanlayıcıları ve Güç Yönetimi

Mobil işletim sistemleri, enerji tasarrufu ve pil ömrünü korumak amacıyla arka plan işlemlerini sıkı şekilde kontrol eder. CPU frekans ölçekleme, arka plan kısıtlamaları, alarm zamanlaması ve foreground servis politikaları bu denetimin temel bileşenleridir. Android ve iOS farklı mekanizmalara sahip olsa da ortak hedef, gereksiz enerji tüketimini azaltmaktır.

CPU Frekans Yöneticisi (CPU Governor)

Mobil SoC'lerde CPU frekansı dinamik olarak ölçeklenir. Android Linux çekirdeği, farklı governor stratejileri kullanır:

- **ondemand:** Yük arttığında frekansı hızla yükseltir; klasik yaklaşım.
- **interactive:** Daha agresif ve kullanıcı etkileşimine duyarlı; modern cihazlarda yaygındır.
- **schedutil:** CPU scheduler ile entegre; frekans kararları işlem yoğunluğuna göre yapılır.
- **powersave / conservative:** Enerji tasarrufu önceliklidir; düşük seviyeli cihazlarda tercih edilir.

Bu mekanizma nedeniyle:

- yoğun hesaplama → CPU frekans artışı → pil tüketimi,
- uzun süreli arka plan iş → governor kısıtlaması,
- cihaz ekran kapalıyken → düşük frekans moduna geçiş davranışları görülür.

Android Battery Optimization

Android 6.0 (API 23) ile güç yönetimi daha agresif hale gelmiştir. Başlıca iki mekanizma:

Doze Mode

Cihaz uzun süre hareketsiz ve ekran kapalı kaldığında:

- ağ istekleri ertelenir,
- zamanlayıcılar toplu çalıştırılır (batching),
- wakelock alma kısıtlanır,
- arka plan servisleri durdurulur.

Doze, scheduled işlerin *anında* tetiklenmesini engeller. Bu nedenle Android, alarm'ları ve zamanlanmış işleri toplu ve gecikmeli çalıştırabilir.

App Standby

Uzun süre kullanılmayan uygulamalar *standby bucket* içine alınır.

- Ağ erişimi kısıtlanır,
- Zamanlanmış işler seyrekleştirilir,
- Push mesajı gelse bile execution penceresi daraltılır.

Bu mekanizma gelişmiş güç tasarrufu sağlar ancak zamanlama hassasiyeti gerektiren uygulamalar için dikkat edilmesi gereken bir durumdur.

Alarm Batching ve Throttling

Android, zamanlanmış işlemleri pil tasarrufu için toplu halde çalıştırır.

- **Alarm batching:** Birden fazla alarm aynı zaman penceresinde tetiklenir.
- **Throttling:** Sık tetiklenen alarm'lar OS tarafından geciktirilir.

Örneğin:

- Normal alarm → birkaç saniyelik gecikmeyle tetiklenebilir,
- Doze Mode sırasında *setExact()* bile gecikebilir,
- Yalnızca *setExactAndAllowWhileIdle()* yüksek öncelikli alarm sağlar.

iOS Energy Heuristics

iOS, güç yönetiminde daha katı ve daha deterministiktir. iOS cihazları Apple'ın donanım kontrolü altında olduğundan enerji tüketimi sıkı koşullara bağlıdır.

iOS aşağıdaki durumları enerji açısından riskli olarak değerlendirir:

- çok sık ağ isteği,
- GPS'in gereksiz aktif kalması,
- ekran dışı (background) uzun süreli CPU kullanımı,
- büyük verilerin sıkça işlenmesi,
- sensörlerin sürekli aktif kalması.

iOS enerji sezgisi (*heuristics*) şu davranışları tetikler:

- arka plan uygulamalarını hızla *suspended* konumuna almak,
- background fetch pencerelerini nadir çalıştırmak,
- gereksiz CPU kullanımında uygulamayı throttle etmek,
- aşırı kaynak kullanımında uygulamayı kill etmek.

iOS ve Android Alarm Karşılaştırması

Android

Alarm davranışı:

- gecikmeye açıktır,
- Doze Mode altında toplulaştırılır,
- hassas alarm için özel API gerekir,
- uygulama optimize edilmemiş olsa bile tetiklenir.

iOS

iOS'ta klasik anlamda “alarm API” yoktur. Bunun yerine:

- Local Notification zamanlaması kullanılır,
- OS zamanı geldiğinde bildirimi garantili şekilde tetikler,
- Doze benzeri bir mod yoktur,
- yeniden başlatma sonrası bile tetikleme koruması vardır.

Bu nedenle iOS, scheduled notification konusunda Android'e göre daha deterministik davranır.

Sonuç

İşletim sistemi zamanlayıcıları ve güç yönetimi, arka plan görevlerinin davranışını belirleyen temel mekanizmalardır. Android, Doze Mode ve App Standby ile agresif tasarruf sağlarken; foreground servislere özel bir koruma alanı sunar. iOS daha basit ancak daha sıkı bir model benimser ve arka plan işlemlerini hızla askıya alır. Alarm batching, throttling ve CPU governor stratejileri zamanlanmış bildirimler, konum takibi ve uzun süreli işlemler üzerinde doğrudan etkiye sahiptir.

IPC ve Uygulamalar Arası İletişim (Android ve iOS)

Modern mobil işletim sistemlerinde uygulamalar birbirinden izole edilmiş sand-box ortamlarında çalışır. Bu nedenle uygulamaların birbirleriyle veri paylaşabilmesi için özel iletişim mekanizmaları gereklidir. Android ve iOS bu konuda farklı API'ler ve güvenlik modelleri sunar. Bu bölümde IPC (Inter-Process Communication) mekanizmaları detaylı şekilde incelenmektedir.

Android IPC Modeli

Android, Linux tabanlı bir sistemdir ve temel IPC mekanizması **Binder** altyapısına dayanır. Uygulamalar yalnızca OS tarafından onaylanan yollarla birbiriyle iletişim kurabilir.

Binder IPC Mekanizması

Binder, Android'de tüm yüksek seviye IPC iletişiminin temelidir.

- İşlemler arası veri aktarımı için yüksek performanslı bir kernel modülü sağlar.
- Tüm sistem servisleri Binder üzerinden çalışır (örn. Location Service, Media Service).
- Uygulamalar da Binder üzerinden birbirine bağlanabilir.

Binder'ın özellikleri:

- tip güvenli (type-safe) iletişim,
- düşük kopyalama (zero-copy),
- işlem sınırları boyunca güvenli aktarım,
- yetkilendirme kontrolü.

AIDL (Android Interface Definition Language)

AIDL, Binder üzerinden uygulamalar arası iletişim için kullanılan arabirim tanımlama dilidir.

AIDL kullanımı:

1. Arabirim tanımı yapılır,
2. Sistem arabirimi derler,
3. Uygulamalar binder üzerinden birbirleriyle konuşur.

AIDL tipik olarak:

- medya hizmetleri,
- ödeme servisleri,
- güvenli veri transferi,
- sistem uygulamaları

tarafından kullanılır.

Mobil geliştiricilerin çoğu doğrudan AIDL yazmaz; ancak birçok büyük servis (Google Play, Location Service) AIDL temelinde çalışır.

ContentProvider ve Güvenlik

ContentProvider, Android'in veri paylaşımı için sunduğu en yüksek seviyeli IPC mekanizmasıdır. Uygulamalar arası veri aktarımında kullanılır.

Güvenlik riskleri ve koruma yöntemleri:

- Her ContentProvider bir izin modeli ile korunmalıdır,
- Okuma ve yazma izinleri ayrı tanımlanabilir,
- URI'ler hassas veri sızdırabilir (örn. dosya yolu),
- OS gerektiğinde paylaşımı engeller.

Uygulamalar:

- rehber,
- galeri,
- mesajlaşma uygulamaları

ile veri paylaşırken ContentProvider kullanır.

iOS IPC Modeli

iOS, Android'e göre daha sıkı bir izolasyon modeline sahiptir. iOS uygulamaları biriyle yalnızca Apple tarafından onaylanan özel mekanizmalar üzerinden iletişim kurabilir.

iOS App Groups

App Groups, aynı geliştiriciye ait uygulamaların veri paylaşmasına izin veren bir mekanizmadır.

Özellikleri:

- Ortak bir konteyner dizini oluşturur,
- Widget, extension ve ana uygulama arasında dosya paylaşılır,
- Paylaşılan UserDefaults kullanılabilir,
- Güvenlik App Store profilindeki entitlements ile sağlanır.

App Groups Android'deki "shared storage" kavramının güvenli sürümü olarak düşünülebilir.

Custom URL Schemes

iOS, uygulamaların özel URL şemaları ile birbirini çağırmasına izin verir:

`myapp://open?id=42`

Custom URL Schemes:

- uygulamayı foreground'a getirir,
- parametre taşıyabilir,
- sandbox dışına veri çıkarmadan yönlendirme sağlar.

Güvenlik:

- URL'yi dinleyen herhangi bir uygulama çağrıyı ele alabilir,
- hassas veri custom scheme ile gönderilmemelidir.

iOS Universal Links

Universal Links, web URL'lerini doğrudan uygulamaya yönlendiren modern ve güvenli bir mekanizmadır.

Özellikleri:

- HTTPS zorunludur,
- Domain, Apple ile doğrulanmalıdır,
- Kullanıcı uygulamaya gitmek istiyorsa otomatik yönlendirilir,
- Custom URL schemes'e göre daha güvenlidir.

Universal Links, mobil web ve native uygulama entegrasyonunun temelidir.

Android ve iOS IPC Karşılaştırması

- Android, düşük seviyede Binder ve AIDL ile güçlü IPC yetenekleri sunar; iOS ise daha sınırlı ve kontrollü bir modele sahiptir.
- Android'de veri paylaşımı ContentProvider ile yapılabilirken, iOS'ta yalnızca App Groups aracılığıyla mümkündür.
- iOS Custom URL Schemes, Android'deki "intent filter" kavramına benzer ancak güvenlik açısından daha sınırlıdır.
- Universal Links, hem Android hem iOS'ta web tabanlı yönlendirme için en güvenli yöntemdir.
- iOS sandbox modeli Android'e kıyasla daha katıdır; uygulamalar birbirlerine doğrudan erişemez.

Sonuç

IPC, sandboxing mimarisinin doğal bir uzantısıdır. Android geniş ve esnek bir işlem-lerarası iletişim altyapısı sunarken, iOS güvenliği ön planda tutar ve daha sınırlı mekanizmalar sağlar. Binder, AIDL ve ContentProvider Android'in güçlü IPC araçlarıdır; iOS'ta App Groups, URL Schemes ve Universal Links uygulamalar arası iletişim için kullanılan ana yöntemlerdir. Bu mekanizmaların doğru anlaşılması, güvenli veri paylaşımı ve sağlıklı uygulama mimarisi kurmak için kritik öneme sahiptir.

OS Güvenliği: İşletim Sisteminin Engellediği ve Kontrol Ettiği Mekanizmalar

Mobil işletim sistemleri gizlilik, güvenlik ve veri bütünlüğü sağlamak için uygulamaların çalışma şeklini sıkı bir şekilde denetler. Bu bölümde, Android ve iOS'un donanım ve yazılım seviyesinde sunduğu güvenlik mekanizmaları; Keychain/Keystore yapısı, Secure Enclave, TLS pinning davranışları, ekran görüntüsü korumaları ve pano (clipboard) gizliliği detaylı olarak incelenmektedir.

Android Keystore ve iOS Keychain

Her iki platform da hassas verilerin (token, şifre, özel anahtarlar) güvenli şekilde saklanması için özel bir güvenli depolama birimi sağlar.

Android Keystore

Android Keystore, kriptografik anahtarların sistem tarafından yönetildiği güvenli bir depolama alanıdır.

- Uygulama anahtarın kendisine direkt erişemez,
- Anahtar yalnızca OS üzerinden kullanılır,
- Private key asla uygulama hafızasında görünmez,
- Anahtar belirli güvenlik koşullarıyla sınırlanabilir (örn. ekran kilidi, biometric).

Keystore, cihaz destekliorsa donanım tabanlı güvenli alan *TEE (Trusted Execution Environment)* içinde çalışır.

iOS Keychain

iOS Keychain, şifrelenmiş bir sistem veritabanıdır.

- AES-256 şifrelemeli güvenli depolama,
- Donanım güvenlik modülleriyle entegredir,
- Biometric ile eşleştirilebilir,
- iCloud Keychain ile senkronize edilebilir (isteğe bağlı).

Keychain kayıtları yalnızca ilgili uygulama (veya App Group) tarafından erişilebilir.

Secure Enclave (iPhone)

iOS cihazlarında kriptografik işlemleri güvenli bir şekilde gerçekleştiren bağımsız bir işlemci bulunur: **Secure Enclave**.

Özellikleri:

- Ana CPU'dan tamamen bağımsız çalışır,
- Touch ID / Face ID verilerini işler,
- Private key'leri cihaz dışına çıkarmadan imzalama yapar,
- Kırılması son derece zordur,
- Belleği ve çalışması OS tarafından bile tam olarak okunamaz.

Secure Enclave, Keychain ile entegre şekilde çalışır.

TLS Pinning ve OS Davranışı

Uygulamalar HTTPS trafiğinde kendi SSL sertifikalarını doğrulamak için *TLS pinning* kullanabilir. OS bu işlem üzerinde bazı kontrol mekanizmalarına sahiptir.

Android

- OkHttp, Cronet ve WebView farklı güvenlik politikalarına sahiptir,
- Sertifika pinning yanlış yapılırsa ağ isteği tamamen engellenir,
- Kullanıcı sertifikası eklese bile (MITM için) uygulama bunu reddedebilir,
- Ancak root'lu cihazlarda pinning atlatılabilir.

iOS

iOS daha sıkı bir TLS güvenlik politikasına sahiptir.

- ATS (**A**pp **T**ransport **S**ecurity) zorunludur,
- Weak cipher suite kullanımı yasaktır,
- Pinning başarısız olursa istek kesinlikle başarısız olur,
- iOS sistem sertifika deposu manipüle edilmesi daha zordur.

Pinning OS tarafından desteklenir ancak hatalı yapılandırma bağlantıyı tamamen durdurur.

Screenshot ve Screen Recording Korumaları

Mobil OS'ler uygulama ekran görüntüsü alınmasını veya ekran kaydı yapılmasını sınırlayabilir.

Android

- FLAG_SECURE set edildiğinde ekran görüntüsü alnamaz,
- Recent Apps ekranında thumbnail bulanık görünür,
- Ekran kaydı da engellenir,
- Bu koruma yalnızca uygulama tarafından aktif hale getirilebilir.

iOS

iOS'ta ekran görüntüsü engelleme doğrudan mümkün değildir. Ancak uygulama:

- Ekran kaydı sırasında OS tarafından bilgilendirilir,
- Hassas içerikleri maskeleyebilir (`UIScreen.isCaptured`)
- ReplayKit ile kontrollü ekran kaydı yapılabilir.

iOS kullanıcı kontrolünü esas alır; tam engelleme izin verilmez.

Clipboard (Pano) Gizliliği

Son yıllarda pano erişimi gizlilik riski olarak değerlendirilmiştir.

Android

Android 10+ sürümlerinde:

- Arka plandaki uygulamalar clipboard okuyamaz,
- Sadece foreground'da olduğunda erişim verilir,
- Kullanıcı metin kopyaladığında başka uygulama sessizce alamaz.

iOS

iOS 14 ile pano erişimi denetlenmiştir:

- Uygulama clipboard okuduğunda kullanıcıya bildirim verilir,
- Arka planda pano okuma tamamen engellenmiştir,
- WebView içindeki script'lerin pano okuması sınırlanır.

Clipboard gizliliği hem güvenlik hem kullanıcı şeffaflığı için kritik öneme sahiptir.

OS Seviyesinde Engellenen Uygulama Davranışları

Aşağıdaki işlemler her iki platform tarafından doğrudan engellenir:

- diğer uygulamaların dosyalarına erişmek,
- gizlice kamera veya mikrofon açmak,
- kullanıcıdan habersiz ekran kaydetmek,
- kullanıcı fark etmeden background location almak,
- root/administrator yetkisi almaya çalışmak,
- sistem uygulamalarının verilerine erişmek,
- TLS trafiğini manipüle etmek,
- şifreleme anahtarlarını dışarı çıkarmak.

Bu kısıtlamalar işletim sisteminin güvenlik mimarisinin temel yapı taşlarını oluşturur.

Sonuç

Android ve iOS, güvenlik açısından çok katmanlı bir mimari kullanır. Keystore ve Keychain hassas anahtarları korurken, Secure Enclave donanım seviyesinde güvenlik sağlar. TLS pinning, pano kısıtlamaları, ekran görüntüsü korumaları ve sandboxing gibi mekanizmalar kullanıcı gizliliğini savunur. Bu kısıtlamaları anlamak, “neden bunu yapamıyorum?” sorularının cevaplarını netleştirir ve güvenli uygulama geliştirme için temel oluşturur.

Ağ İletişimi (Gelişmiş Konular): QUIC, Connection Pooling, Ağ Geçişi ve Captive Portal

Modern mobil uygulamalar yalnızca HTTP üzerinden veri alışverişi yapmaz; iletişim performansı işletim sisteminin taşıma katmanı, bağlantı havuzu, ağ geçiş davranışları ve ağ doğrulama mekanizmalarından doğrudan etkilenir. Bu bölümde QUIC/HTTP/3 protokolü, OS seviyesinde bağlantı havuzu yönetimi, Wi-Fi / hücresel ağ geçişi ve captive portal tespiti ayrıntılı olarak ele alınmaktadır.

QUIC ve HTTP/3

HTTP/3, Google tarafından geliştirilen ve daha sonra IETF tarafından standartlaştırılan QUIC protokolü üzerine kuruludur. QUIC, TCP yerine UDP üzerinde çalışan modern bir taşıma katmanıdır.

QUIC'in Temel Özellikleri

- **UDP tabanlıdır:** Bağlantılar UDP üzerinden akış (stream) mantığıyla yürütülür.
- **0-RTT bağlantı kurulumu:** Tekrarlayan bağlantılar için sunucuya istek hemen gönderilebilir.
- **Head-of-Line Blocking yoktur:** Tek bir kayıp paket tüm akışı durdurmaz (TCP'deki en büyük darboğaz çözülmüştür).
- **TLS 1.3 şifrelemeyi protokolün içine gömülüdür:** TLS ve taşıma katmanı birleşmiştir.
- **Daha iyi çoklu akış desteği:** Aynı bağlantı üzerinde paralel istekler birbirini engellemez.

Mobil Uygulamalar Açısından Avantajları

- Düşük gecikme (RTT azaltımı),
- Paket kaybında hızlı toparlanma,
- Zayıf hücresel bağlantılarda daha stabil çalışma,
- TCP bağlantı kuyruğu etkilerinin azalması.

Android: Modern Chromium tabanlı *Cronet* kütüphanesi QUIC destekler.
iOS: *NSURLSession* HTTP/3 desteğini iOS 15+ sürümlerinde sunar.

OS-Level Connection Pooling

Hem Android hem iOS, uygulamaların ağ bağlantılarını daha verimli yönetebilmek için bağlantı havuzu (*connection pool*) kullanır.

Temel Prensipler

- Aynı host'a yapılan isteklerde bağlantı yeniden kullanılabilir,
- TCP/TLS handshake maliyeti azaltılır,
- Paralel HTTP istekleri için sınırlı sayıda socket kullanılır.

Android

Android'in bağlantı havuzu OkHttp veya Cronet tarafından yönetilir.

- Havuz zaman aşımı genellikle 5 dakika civarındadır,
- Aynı anda açılacak paralel bağlantı sayısı sınırlıdır,
- Keep-alive olmayan sunucular bağlantı yenilemeye zorlar.

iOS

iOS `NSURLSession` havuzu sistem seviyesinde yönetir.

- Oturumlar (session) bağlantı havuzlarını paylaşabilir,
- Arka plan işlerinde bağlantılar askıya alınabilir,
- iOS enerji heuristics bağlantıları erken kapatabilir.

Sonuç: Bağlantı havuzu davranışı performansı ciddi şekilde etkiler; gereksiz yeni bağlantı başlatmak gecikmeyi artırır.

Cellular / Wi-Fi Switching

Mobil cihazlar ağ bağlantısı tiplerini dinamik olarak değiştirebilir. Bu süreç, uygulama seviyesinde kesintilere yol açabilir.

Switching Davranışı

- Wi-Fi sinyali zayıfladığında cihaz otomatik olarak hücresel ağa geçer,
- IP adresi değişir,
- Mevcut TCP bağlantıları kopar ve yeniden kurulmalıdır.

QUIC ile İyileşme

QUIC'in en büyük avantajlarından biri **IP adresi değişse bile bağlantıyı sürdürebilmesidir**. Buna *connection migration* denir.

- TCP bağlantısı ağ değişiminde kopar,
- QUIC bağlantısı koptuğu yerden devam eder.

Bu özellik özellikle:

- VOIP,
- canlı yayın,
- gerçek zamanlı mesajlaşma

için büyük avantaj sağlar.

Captive Network Detection

Captive portal, otellerde, havaalanlarında veya kafelerde görülen ve internet erişimi öncesinde kimlik veya onay isteyen ağ türüdür.

Android

Android, özel bir sorgu ile ağın captive portal olup olmadığını tespit eder:

- `connectivitycheck.gstatic.com` gibi test endpoint'leri,
- HTTP 204 beklenen yanıt,
- Yanıt gelmezse “Sign-in to Wi-Fi” bildirimini gösterir.

Uygulamalar açısından sonuç:

- DNS çözülebilir ama internet yok olabilir,
- HTTPS istekleri bloke olabilir,
- Arka plan görevleri başarısız olabilir.

iOS

iOS da benzer şekilde captive portal kontrolü yapar:

- `captive.apple.com` üzerinden test,
- Portal tespit edildiğinde **Captive Network Assistant** ekranı açılır,
- Uygulama HTTP isteklerinde SSL hataları alabilir.

Her iki platformda captive portal durumunda kullanıcı müdahalesi olmadan internet erişimi garanti edilemez.

Sonuç

QUIC/HTTP/3 modern taşıma katmanının temelini oluşturur ve mobil ağlarda gecikme ve paket kaybı problemlerine önemli çözümler sunar. OS-level connection pooling hem performans hem enerji optimizasyonu açısından kritiktir. Wi-Fi / hücresel ağ geçişi TCP bağlantıları kesebilirken QUIC bağlantıyı sürdürebilir. Captive network tespiti, uygulamaların “internet var gibi görünüyor fakat aslında yok” durumlarını doğru yorumlaması için zorunlu bir mekanizmadır.

Grafik ve Render Pipeline: CPU/GPU Ayrımı, Overdraw, RenderThread ve Core Animation

Mobil cihazlarda kullanıcı arayüzünün çizilmesi çok aşamalı bir işlem olup CPU, GPU, bellek bant genişliği ve OS zamanlayıcılarının ortak çalışmasını gerektirir. Bu bölümde Android ve iOS'ta render süreci, overdraw, layer compositing, RenderThread ve Core Animation pipeline ayrıntılı şekilde incelenmektedir.

CPU Rendering ve GPU Rendering

Bir UI çerçevesi çizilirken hem CPU hem GPU önemli roller üstlenir.

CPU Görevleri

- Layout hesaplamaları (measure & layout),
- View hiyerarşisi işlenmesi,
- Animasyon değerlerinin hesaplanması,
- Draw komutlarının hazırlanması,
- Bitmap manipülasyonu,
- JS tarafında stil hesaplamaları (React Native).

CPU, UI bileşenlerinin *nasıl* çizileceğini belirler.

GPU Görevleri

- Hazırlanan draw komutlarını işlemek,
- Katmanları (layers) birleştirmek (compositing),
- Transform, opacity, mask, clip gibi işlemleri hızlandırmak,
- Ekrana rasterize etmek.

GPU, UI bileşenlerini *ekrana fiziksel olarak çizer*.

Sonuç: CPU yavaşsa → layout janki, animasyon takılır. GPU yavaşsa → compositing janki, kare atlamaları oluşur.

Overdraw: Aynı Pikseli Birden Fazla Çizmek

Overdraw, bir pikselin gereksiz şekilde birden çok kez çizilmesidir.

Örnek:

- Arka plan → üzerine Card → üzerine bir View → üzerine opacity view.

Bir piksel 3–4 kez yeniden çizilebilir.

Aşırı overdraw:

- GPU yükünü artırır,
- Pil tüketimini yükseltir,
- Kaydırma sırasında jank üretir.

Android GPU Profiler bu bölgeleri renklendirir:

- Mavi — düşük overdraw,
- Yeşil — orta seviye,
- Sarı — yüksek,
- Kırmızı — çok yüksek (kritik jank nedeni).

Layer Compositing

Modern UI sistemleri çizimi tek aşamada yapmaz; UI bileşenleri katmanlara (layer) ayırılır ve GPU bu katmanları birleştirerek final görüntüyü üretir.

Katmanlar:

- ana view katmanı,
- animasyon katmanları,
- opacity/transform katmanları,
- metin katmanları,
- video katmanları.

Aşırı layer sayısı:

- GPU memória tüketimini artırır,
- Compositing süresini uzatır,
- 60 FPS hedefini bozar.

İdeal durumda:

- yalnızca animasyonlu view'ler kendi layer'ına sahip olmalı,
- statik içerikler tek layer'da birleştirilmelidir.

Android RenderThread Mimarisi

Android 5.0 ile birlikte render pipeline üç ayrı thread'e bölünmüştür:

- **UI Thread** Layout, measure, input event'ler, animasyon hesaplamaları.
- **RenderThread** Draw komutlarını GPU'ya hazırlar; CPU tarafında çalışır.
- **GPU Thread** RenderThread'in hazırladığı komutları işler.

Çalışma akışı:

1. UI thread → view hiyerarşisini hesaplar,
2. UI thread → RenderThread'e draw komutlarını gönderir,
3. RenderThread → GPU'ya komutları iletir,
4. GPU → çerçeveyi rasterize eder.

Bu yapı sayesinde UI thread meşgul olsa bile GPU işlemleri gecikmez.

Jank Nedenleri

- UI thread 16 ms içinde layout'u bitiremez → frame düşer,
- RenderThread aşırı draw komutu alır → GPU yetişemez,
- Overdraw → GPU bant genişliğini boğar,
- Büyük bitmap decode → UI thread bloklanır.

iOS Core Animation Pipeline

iOS'ta UI çizimi **Core Animation** tarafından yönetilir. Android'den farklı olarak çoğu işlem GPU optimize edilmiş katmanlarda gerçekleşir.

Pipeline aşamaları:

1. **Layout Pass** AutoLayout constraint çözümleri, view ölçümleri.
2. **Display Pass** Her view kendi *CALayer*'ına çizim yapar.
3. **Compositing Pass** GPU tüm layer'ları birleştirerek final görüntüyü üretir.
4. **VSync** Kare zamanlaması (60 Hz / 120 Hz).

Core Animation'ın avantajları:

- katman tabanlı animasyonlar CPU'yu yormaz,
- opacity, transform, shadow gibi özellikler GPU tarafında çalışır,
- layout ve paint aşaması yalnızca gerektiğinde yapılır.

Bu nedenle iOS animasyonları genellikle daha pürüzsüzdür.

16 ms Kuralı ve Frame-Loss

60 FPS için her frame'nin çizilmesi gereken süre:

$$16.67\text{ ms}$$

Bu süre aşılsa:

- frame atlanır (dropped frame),
- jank ortaya çıkar,
- kaydırma ve animasyon takılır.

60 FPS $\rightarrow$ 16 ms 120 FPS $\rightarrow$ 8.3 ms

Her işlem bu zaman penceresinin içinde bitmelidir.

React Native Özel Durumlar

React Native pipeline'ı üç thread içerir:

- JS Thread — business logic, state, animation hesaplamaları,
- Shadow Thread — layout (Yoga),
- UI Thread — draw komutlarının işlenmesi.

Jank nedenleri:

- JS thread'de ağır loop,
- Bridge üzerinden büyük JSON geçişleri,
- Çok sık setState,
- FlatList 10.000 eleman $\rightarrow$ layout patlaması,
- Ana thread'de ağır iş.

Screenshot:

JS thread 5 ms + Layout 7 ms + GPU 7 ms = 19 ms $\rightarrow$ frame dropped.

Sonuç

Grafik rendering pipeline hem CPU hem GPU için sıkı zamanlama gerektirir. CPU tarafındaki layout ve draw hesaplamaları, GPU tarafındaki layer compositing ve overdraw yönetimi ile birleştğinde performans sınırları ortaya çıkar. Android RenderThread ve iOS Core Animation farklı mimariler kullansa da amaç aynıdır: 16 ms içinde bir kareyi üretmek. Bu süre aşıldığında jank oluşur ve kullanıcı deneyimi bozulur.

Concurrency ve Parallelism: İşletim Sistemi Temel Kavramları

Mobil işletim sistemlerinde uygulama davranışını belirleyen en önemli mimarilerden biri eşzamanlılık (concurrency) ve paralellik (parallelism) yapılarıdır. Bu kavramlar CPU çekirdekleri, iş parçacıkları (threads), işletim sistemi zamanlayıcıları ve context switch gibi düşük seviyeli mekanizmalarla birlikte çalışır. Bu bölümde OS tabanlı concurrency modeli detaylı şekilde açıklanmaktadır.

Concurrency (Eşzamanlılık)

Concurrency, birden fazla görevin aynı zaman aralığında ilerlemesi demektir. Burada görevler gerçekten aynı anda çalışmak zorunda değildir; işletim sistemi görevler arasında hızla geçiş yaparak eşzamanlılık hissi yaratır.

Temel Özellikler

- Tek çekirdekli CPU'da bile mümkündür,
- Görevler sırayla küçük zaman dilimlerinde çalıştırılır,
- Amaç birden fazla işin “ilerlemesini” sağlamaktır,
- Gerçek fiziksel paralellik gerektirmez.

Örnek:

- Kullanıcı kaydırma yaparken ağ isteği devam edebilir,
- JS thread animasyon hesaplamaları yaparken bir yandan event'ler işlenebilir.

Concurrency, mobil uygulamalarda “UI donmadı ama çok iş yapıyor” hissini sağlayan mekanizmadır.

Parallelism (Paralellik)

Parallelism, birden fazla görevin aynı anda, fiziksel olarak farklı CPU çekirdeklerinde çalışmasıdır.

Temel Özellikler

- Çok çekirdekli CPU gerektirir,
- Görevler aynı anda işlenir,
- Performans artışı sağlar,
- CPU yoğun işlerde önemlidir.

Örnek:

- Bir çekirdek JSON parse ederken, diğeri görüntü decode edebilir,
- Android RenderThread ve UI thread paralel çalışır,
- iOS Core Animation pipeline CPU ve GPU'nun paralel çalışmasına izin verir.

Concurrency = mantıksal eşzamanlılık, Parallelism = fiziksel eşzamanlılık.

Process (Süreç) ve Thread (İş Parçacığı)

Process

Bir uygulamanın yürütüldüğü, kendi adres alanına sahip bağımsız bir çalışma birimidir.

Özellikleri:

- Her process'in belleği diğerinden ayrıdır,
- Crash olduğunda diğer süreçleri etkilemez,
- IPC (inter-process communication) gerektirir,
- Android'de her uygulama varsayılan olarak bir process'te çalışır.

Thread

Bir process içinde çalışan hafif bir yürütme birimidir.

Özellikleri:

- Tüm thread'ler aynı memory space'i paylaşır,
- Context switching çok hızlıdır,
- Veri yarışlarına (race condition) açıktır,
- UI işlemleri genellikle tek bir UI thread'de yapılır.

Context Switch (Bağlam Değişimi)

Context switch, işletim sisteminin CPU'yu bir iş parçasından (thread) diğerine aktarmasıdır. Bu işlem, çok görevli (multitasking) işletim sistemlerinin temel özelliğidir. Mobil cihazlarda (Android/iOS) yoğun context switch genellikle performans ve enerji tüketimi üzerinde doğrudan etki yaratır.

Context Switch Sırasında Yaşanan Adımlar

Bir thread durdurulup diğerine geçilirken işletim sistemi aşağıdaki işlemleri gerçekleştirir:

1. **CPU register'larının kaydedilmesi** Thread'in anlık durumu (program counter, stack pointer, genel register'lar) kernel'de saklanır.
2. **Stack durumunun korunması** Mevcut thread'in stack'i kendi süreç kontrol bloğunda (TCB) işaretlenir.
3. **Yeni thread'in register'larının yüklenmesi** Scheduler tarafından seçilen thread'in kaydedilmiş register değerleri CPU'ya geri yüklenir.
4. **CPU cache line yenilenmesi** Yeni thread'in çalışacağı veri ve talimatlar genellikle mevcut cache'de bulunmaz, bu nedenle cache kısmen temizlenir. Cache miss, ek gecikmeye neden olur.
5. **Memory barrier'ların uygulanması** Paylaşılan hafıza yapılarında veri tutarlılığı sağlanır.
6. **Program counter'ın yeni thread'e yönlendirilmesi** CPU artık yeni thread'in kaldığı yerden devam eder.

Bu döngü milisaniyenin çok küçük bir bölümünde gerçekleşse de yoğun context switch sistem genelini yavaşlatabilir.

Neden Context Switch Yapılır?

- OS scheduler yüksek öncelikli bir thread'i çalıştırmak ister,
- UI thread'e öncelik verilmesi gerekir,
- Bir thread uykuya geçer (I/O, network bekler),
- Bir thread zaman dilimini (time slice) tüketir,
- Bir interrupt (kesme) meydana gelir,
- Arka plan işler tetiklenir (GC, WorkManager, iOS background task).

Context Switch'in Maliyetleri

Context switch donanımsal ve yazılımsal maliyetler içerir:

- **CPU maliyeti:** Register ve stack durumunun sürekli kaydedilip yüklenmesi.
- **Cache maliyeti:** *Cache pollution* oluşur; yeni thread'in verileri cache dışındadır, bu da CPU'nun RAM'den veri çekmesine neden olur.
- **Enerji maliyeti:** Mobil CPU'lar düşük güç moduna kolayca geçer; yoğun thread değişimi CPU'yu sürekli uyanık tutar.
- **Latency maliyeti:** UI ile ilgili thread planlandığı anda çalışamazsa jank oluşur.

Aşırı Context Switch'in Sonuçları

Aşırı context switching şu problemlere yol açar:

- **Jank (frame drop):** UI thread zamanında çalıştırılmaz, çerçeve süresi 16ms eşiğini aşar.
- **Pil tüketimi artışı:** CPU derin uyku (deep sleep) moduna geçemez, sürekli "wakeup" yaşanır.
- **Scheduler yoğunluğu:** OS zamanlayıcısı daha fazla iş yapmak zorunda kalır.
- **Thread starvation:** Düşük öncelikli thread'ler hiç çalıştırılmayabilir.
- **Lock-contention artışı:** Paylaşılan yapılarda kilit bekleme süreleri uzar.

Android ve iOS Perspektifinde Context Switch

Android (Linux CFS Scheduler)

- Çok sayıda thread oluşturmak yaygındır, bu da context switch sayısını artırır.
- ART runtime tarafından başlatılan GC thread'leri yoğun switching yaratabilir.
- RenderThread ve UI Thread arasında sürekli geçiş olur.
- WorkManager arka planda zamanlanmış işler ile düzenli wakelock tetikler.

iOS (GCD + Preemptive Scheduler)

- GCD (Grand Central Dispatch), gereksiz thread oluşturmayı azaltarak context switch maliyetini düşürür.
- Dispatch queues sayesinde iş yükü daha kontrollü dağılır.
- Core Animation pipeline CPU ve GPU işlerini paralel optimize eder.

Mobil Geliştirici İçin En Kritik Sonuç

- Gereksiz thread oluşturmak performans kaybıdır.
- UI thread bloklanırsa OS öncelik artırsa bile çerçeve yetişmez.
- React Native'de JS thread bloklandığında context switch olsa bile UI durur.
- Ağ, disk, JSON parse işlemleri mutlaka background queue/thread'de yapılmalıdır.

Multithreading ve Multitasking

Mobil işletim sistemleri hem çoklu iş parçacığı (multithreading) hem de çoklu görev (multitasking) modellerini kullanarak aynı anda birden fazla işin ilerlemesini sağlar. Bu iki kavram genellikle karıştırılır; ancak işleyişleri ve maliyetleri farklıdır.

Multithreading

Multithreading, bir **aynı process** içerisinde birden fazla iş parçacığının (thread) paralel veya eşzamanlı çalışmasıdır. Tüm thread'ler aynı adres alanını ve aynı dosya tamtıcılarını paylaşır.

Temel Özellikler

- Tüm thread'ler aynı bellek alanını paylaşır,
- Thread oluşturma maliyeti düşüktür,
- Veri yarışları (race condition) ve kilitlenme (deadlock) riski yüksektir,
- İşletim sistemi thread'ler arasında context switch yapar,
- UI thread genellikle diğerlerinden ayrı tutulur.

Mobil Uygulamalardaki Rolü

- UI thread (main thread): kullanıcı etkileşimi ve çizim,
- Background thread: ağ, disk, JSON parse, CPU yoğun işler,
- RenderThread (Android): GPU komutlarını işler,
- JS Thread (React Native): business logic ve state yönetimi,
- iOS GCD queue'ları: concurrency yönetimini otomatik optimize eder.

Multithreading'in Riskleri

- Paylaşılan objeler yanlışlıkla iki thread'de aynı anda güncellenebilir,
- Yanlış kilit tasarımı deadlock oluşturabilir,
- Fazla thread → fazla context switch → jank,
- React Native'de JS thread bloklanırsa UI tamamen donar.

Multitasking

Multitasking, işletim sisteminin **birden fazla process'i** aynı zaman aralığında çalıştırmasıdır. Process'ler birbirinden tamamen izole edilmiştir (sandboxing).

Temel Özellikler

- Her process'in kendine ait bellek alanı vardır,
- Process'ler birbirinin RAM'ine erişemez,
- Her process'in kendi iş parçacıkları vardır,
- Process başlatma maliyeti thread'e göre çok daha yüksektir,
- IPC mekanizmaları gerekir (Binder, AIDL, App Groups, XPC).

Mobil İşletim Sistemlerinde Multitasking

Android

- Her uygulama varsayılan olarak kendi process'inde çalışır,
- Activity, Service, BroadcastReceiver aynı process'i paylaşır,
- OS düşük bellek durumunda “cached” process'leri öldürür.

iOS

- Uygulamalar arka planda yalnızca sınırlı süre çalışır,
- Bellek uyarısı alındığında uygulama askıya alınabilir,
- Arka plan modları yalnızca belirli kategoriler için izinlidir (müzik, VoIP, konum, sağlık sensörleri).

Multithreading ve Multitasking Arasındaki Farklar

Özellik	Multithreading	Multitasking
Çalışma alanı	Aynı bellek alanı	Ayrı bellek alanları
Birim	Thread	Process
Yaratma maliyeti	Düşük	Yüksek
İletişim	Paylaşılan bellek	IPC gerekir (Binder, pipes)
Risk	Race condition, deadlock	Yok (tam izolasyon)
OS davranışı	Thread'leri zamanlar	Process'leri öldürebilir
Mobil kullanım	UI + background iş	Birden fazla uygulama

Gerçek Mobil Örnekler

Multithreading Örnekleri

- Bir uygulama hem animasyon oynatır hem ağ isteği yapar,
- Android RenderThread UI thread ile paralel çalışır,
- iOS GCD background queue'da JSON parse eder,
- React Native'de JS Thread + Shadow Thread + UI Thread.

Multitasking Örnekleri

- X arka planda müzik çalarken Y açıktır,
- WhatsApp arka planda beklerken kamera uygulaması çalışabilir,
- Android OS bellek yetmediğinde arka plandaki bir process'i öldürür.

Geliştirici Açısından Kritik Sonuçlar

- UI thread yalnızca hafif işler için kullanılmalıdır,
- Ağ ve disk işlemleri mutlaka background thread'lerde çalıştırılmalıdır,
- Thread sayısını artırmak her zaman performansı artırmaz,
- Çok fazla thread → fazla context switch → pil tüketimi,
- Multitasking OS tarafından kontrol edildiği için uygulama her an öldürülebilir,
- Arka planda çalışacak işler Foreground Service (Android) veya Background Modes (iOS) ile bildirilmelidir.

Sonuç

Multithreading, aynı uygulama içinde birden çok iş parçacığının çalışmasını sağlarken; multitasking, birçok uygulamanın aynı cihazda eşzamanlı bulunmasını mümkün kılar. Mobil işletim sistemleri bu iki kavramı farklı kurallar, limitler ve güvenlik önlemleriyle yönetir. Bu yapıların doğru anlaşılması, yüksek performanslı ve enerji verimli uygulamalar geliştirmek için hayati öneme sahiptir.

OS Scheduler (Zamanlayıcı)

Android Linux çekirdeğinin CFS (Completely Fair Scheduler) zamanlayıcısını kullanır; iOS ise GCD (Grand Central Dispatch) ve pthread tabanlı bir scheduler kullanır.

Scheduler Görevleri

- thread'lere CPU zamanı ayırmak,
- yüksek öncelikli işleri öne almak,
- düşük öncelikli işleri geciktirmek,
- enerji optimizasyonu sağlamak,
- UI thread'e öncelik vermek.

Android'de:

- UI thread'e yüksek öncelik,
- Background thread'lere düşük öncelik,
- Foreground service process'leri daha geç öldürülür.

iOS'ta GCD kuyruğa (queue) dayalı bir concurrency modeli kullanır.

Synchronous vs Asynchronous

Synchronous

- İş tamamlanana kadar beklenir,
- UI thread üzerinde kullanmak risklidir,
- Ağ isteklerinde ANR üretir.

Asynchronous

- İş arka planda yapılır,
- Sonuç callback/future/promise ile geri gelir,
- Mobil uygulamalarda temel modeldir.

Async modeller:

- Java/Kotlin: coroutines,
- Swift: async/await,
- JS/TS: Promise/async/await.

Race Condition ve Deadlock

Race Condition

Birden fazla thread aynı veriyi eşzamanlı değiştirirse tutarsızlık oluşur.

Örnek:

- React Native’de aynı state’i birden fazla async fonksiyon güncelliyorsa.

Deadlock

İki thread birbirini bekleyerek kilitlenir ve işlem durur.

Örnek:

- iOS’ta main queue üzerinde senkron çağrı yapmak,
- Android’de synchronized blokların dairesel bağımlılığı.

Mobil Geliştirici İçin Kritik Sonuçlar

- UI thread daima serbest tutulmalıdır,
- Ağ, JSON parse, şifreleme, dosya işlemleri her zaman background’da yapılmalıdır,
- Paralellik CPU-bound işlerde performansı çarpıcı şekilde artırır,
- Aşırı thread oluşturmak jank üretir,
- React Native’de JS thread bloklanırsa UI tamamen donar,
- iOS’ta AutoLayout ağır layout hesaplamalarında büyük gecikme yaratabilir.

Sonuç

Concurrency, görevlerin aynı zaman aralığında ilerlemesini; parallelism ise fiziksel olarak aynı anda çalışmasını sağlar. İşletim sistemi zamanlayıcıları bu iki modeli yönetir ve thread’ler arasında context switch yaparak CPU kaynaklarını dağıtır. Bu mekanizmaların doğru anlaşılması, mobil uygulamalarda performans optimizasyonu, jank azaltımı ve enerji verimliliği için kritiktir.

Mobil İşletim Sistemlerinde Concurrency ve Threading Kavramları (Özet)

Aşağıdaki kavramlar Android ve iOS'ta eşzamanlılık, paralellik, bellek yönetimi ve performans optimizasyonunun temel taşlarını oluşturur.

1. Thread

Bir process içinde çalışan hafif yürütme birimidir. Tüm thread'ler aynı bellek alanını paylaşır.

2. Main/UI Thread

UI güncellemelerinin, input event'lerinin ve animasyonların işlendiği ana iş parçacığıdır. Bloklanmaması gerekir.

3. Background Thread

Ağ istekleri, dosya IO, JSON parse gibi ağır işler için kullanılan arka plan iş parçacığıdır.

4. Worker Thread

Önceliği düşük, cpu-bound veya io-bound görevler için tahsis edilen iş parçacığıdır.

5. Daemon Thread

Arka planda çalışan ve uygulama kapanınca sonlandırılan hafif iş parçacıklarıdır.

6. Thread Priority

OS scheduler, yüksek öncelikli thread'lere daha fazla CPU zamanı ayırır. UI thread en yüksek önceliğe sahiptir.

7. Thread Starvation

Düşük öncelikli bir thread'in uzun süre CPU alamaması durumudur.

8. Thread Safety

Paylaşılan veriye birden fazla thread'in aynı anda erişmesinin güvenli olmasıdır. Atomic yapı ve kilit mekanizmaları gerektirir.

9. Reentrant Kod

Bir fonksiyonun aynı anda birden çok thread tarafından çağrılabilir olmasıdır; global durum kullanmaz.

10. Thread Pool

Önceden oluşturulmuş thread grubudur. Yeni thread yaratmadan işleri hızlıca tahsis eder, enerji ve performans kazandırır.

11. Fixed Thread Pool

Sabit sayıda iş parçacığına sahip havuzdur. CPU yoğun işler için idealdir.

12. Cached Thread Pool

İhtiyaç duyuldukça thread oluşturan, boşta kalanları serbest bırakan havuzdur. Çok sayıda kısa ömürlü iş için uygundur.

13. Single Thread Executor

Tüm işleri tek bir sırada çalıştırır. Sıralı işlem gerektiren durumlarda kullanılır.

14. Work Stealing Pool

İş yükünü otomatik dengeleyen havuz türüdür. Paralel algoritmalarda yüksek verim sağlar.

15. Concurrency

Birden fazla görevin aynı zaman aralığında ilerlemesidir. Gerçek paralellik gerektirmez.

16. Parallelism

Birden fazla görevin fiziksel olarak aynı anda çalışmasıdır. Çok çekirdekli CPU gerektirir.

17. Preemptive Multitasking

OS scheduler gerektiğinde bir thread'i durdurup başka bir thread'i çalıştırabilir.

18. Cooperative Multitasking

Thread'ler CPU'yu kendi isteğiyle bırakır. Mobil OS'larda nadirdir.

19. Synchronous Execution

İş tamamlanana kadar bekleyen bloklayıcı çalışmadır. UI thread için tehlikelidir.

20. Asynchronous Execution

İş arka planda yapılır, sonuç callback veya future ile döner. Mobil dünyada temel modeldir.

21. Futures ve Promises

Asenkron işlemlerin gelecekte tamamlanacağına dair temsilcilerdir.

22. Event Loop

JavaScript'te olayları ve callback'leri sırayla işleyen döngüdür. React Native JS thread bu modeli kullanır.

23. Process

Kendi bellek alanına sahip bağımsız çalışma birimi. Uygulamalar process olarak çalışır.

24. Process Isolation (Sandboxing)

Bir uygulamanın diğerinin hafızasına veya dosyalarına erişememesini sağlayan güvenlik modeli.

25. IPC (Inter-Process Communication)

Process'ler arası iletişim modelleri: Binder, AIDL, sockets, XPC, App Groups.

26. Shared Memory

Process'ler arasında paylaşılan RAM alanı. Mobil OS'lerde sıkı şekilde sınırlandırılmıştır.

27. Message Passing

Thread'ler ve process'ler arası veri aktarımının kopyalanarak yapılmasıdır.

28. OS Scheduler

Thread'lere CPU zamanı tahsis eden mekanizma. Android CFS, iOS GCD üzerine kuruludur.

29. Time Slice (Quantum)

Her thread'in CPU üzerinde çalışması için verilen küçük zaman dilimidir.

30. Priority Scheduling

Yüksek öncelikli thread'ler düşük önceliklilerden önce çalıştırılır.

31. Real-Time Threads

Kesin zaman garantisi gereken yüksek öncelikli işler için kullanılır.

32. CPU Affinity

Bir thread'in belirli bir CPU çekirdeğine sabitlenmesidir. Mobil OS'lerde genelde otomatik yapılır.

33. Context Switch

CPU'nun bir thread'den diğerine geçerken register, stack ve cache durumunu değiştirmesi. Aşırı jank üretir.

34. Kernel Mode vs User Mode

OS çekirdeğinin çalıştığı ayrı erişim seviyeleri. Context switch bu seviyeler arasında geçiş içerebilir.

35. Mutex

Tek seferde yalnızca bir thread'in erişimine izin veren kilittir.

36. Semaphore

Kaynak sayısına göre birden fazla thread'in erişimine izin veren kilit mekanizmasıdır.

37. Spinlock

Thread'in kilidi beklerken uyumadığı, aktif olarak döndüğü kilit tipi. Mobilde enerji tüketimi nedeniyle kaçınılır.

38. Monitor / synchronized

Java/Kotlin'de obje tabanlı kilit yapısıdır. Bloklayıcıdır.

39. Atomic Değişken

Thread-safe artırım, azaltma ve okuma gibi işlemler sağlayan özel CPU talimatlarıyla çalışan değişkenlerdir.

40. Volatile

Değerin her okunduğunda RAM'den çekildiğini garanti eder; thread görünürlüğü sağlar.

41. Memory Barriers

CPU'nun farklı thread'ler arasındaki bellek görünürlüğünü garanti eden mekanizmalardır.

42. Race Condition

Birden fazla thread'in aynı veriyi aynı anda değiştirmeye çalışmasından kaynaklanan tutarsızlık.

43. Deadlock

İki veya daha fazla thread'in birbirini bekleyerek sistemin tamamen kilitlenmesi.

44. Livelock

Thread'ler aktif olarak çalışır ancak iş ilerlemez.

45. Priority Inversion

Düşük öncelikli thread'in, yüksek öncelikli bir thread'i engellemesidir. Android Inversion çözümü uygular.

46. Data Race

Bir veriye aynı anda hem okuma hem yazma yapılması.

47. False Sharing

Aynı cache satırını paylaşan bağımsız değişkenler nedeniyle performans kaybı oluşması.

48. Coroutines (Android)

Suspend fonksiyon tabanlı hafif concurrency modelidir. Thread oluşturmaz.

49. Dispatch Queues (iOS)

GCD tarafından yönetilen paralel iş kuyruqlarıdır. Thread yönetimini otomatikleştirir.

50. OperationQueue (iOS)

Bağımlılık ilişkileri kurulabilen gelişmiş asenkron iş yönetim sistemi.

51. React Native JS Thread

Tüm JS kodunun çalıştığı tek thread. Bloklanırsa UI donar.

52. React Native Shadow Thread

Yoga layout hesaplamalarının yapıldığı thread.

53. React Native UI Thread

Gerçek çizim işlemlerinin Android/iOS tarafında gerçekleştiği thread.

54. RenderThread (Android)

GPU'ya draw komutlarını gönderir; UI thread'den ayrılır.

55. Display Compositor / GPU Thread

Katmanları birleştirerek ekrana çizen GPU pipeline'ıdır.

56. Garbage Collector

Heap üzerinde kullanılmayan nesneleri otomatik temizler. Stop-the-world duraklamaları jank üretebilir.

57. ARC (iOS)

Automatic Reference Counting. Objelerin sahiplik ilişkisine göre bellek yönetimi yapılır.

58. Heap Fragmentation

Dağınık bellek tahsisi performansı düşürür; büyük objeler sığmayabilir.

59. Stack vs Heap

Stack hızlı ve küçük, heap geniş ve yavaştır. Dinamik nesneler heap'te tutulur.

60. Battery Optimization

Doze, App Standby, wakelock ve foreground service gibi enerji yönetimi mekanizmaları mobil uygulamaları doğrudan etkiler.

Process ve Thread Arasındaki Bellek ve CPU Paylaşımı

Bu bölüm, işletim sistemi düzeyinde *process* (süreç) ve *thread* (iş parçacığı) kavramlarının bellek (memory) ve işlemci (CPU) paylaşımı açısından yapısal farklarını resmi ve teknik bir dille özetlemektedir.

Process (Süreç)

Bir process, işletim sistemi tarafından izole edilmiş bağımsız bir yürütme birimidir.

- **Bellek Paylaşımı:** Process'ler bellek alanlarını *paylaşmaz*. Her process'in kendi adres uzayı, heap'i, global değişkenleri ve veri segmenti birbirinden tamamen ayrıdır. İşletim sistemi bu ayrımı "memory isolation" ilkesiyle garanti eder.
- **CPU Paylaşımı:** Process'ler CPU'yu zaman tabanlı *paylaşırlar*. İşletim sistemi, zamanlayıcı (scheduler) aracılığıyla her process'e işlemci süresi tahsis eder (time slicing).
- **Yalıtım:** Yüksek düzeyde izolasyon sunar; bir process'in çökmesi diğer process'leri doğrudan etkilemez.

Özet:

$$Memory = Ayrı, \quad CPU = Paylaşılan$$

Thread (İş Parçacığı)

Thread, aynı process içerisindeki hafif yürütme birimidir ve process kaynaklarını ortak kullanır.

- **Bellek Paylaşımı:** Aynı process'e ait tüm thread'ler ortak bir heap, global değişken alanı ve kod segmenti kullanır. Ancak her thread'in kendi bağımsız *stack* alanı bulunur.
- **CPU Paylaşımı:** Thread'ler de CPU'yu paylaşır; scheduler tarafından ayrı ayrı zamanlanır.
- **Yalıtım:** Düşük izolasyon seviyesine sahiptir. Bir thread'in hatası genellikle aynı process içindeki tüm thread'leri etkiler.

Özet:

$$Memory = Paylaşılan(heap) + Ayrı(stack), \quad CPU = Paylaşılan$$

Karşılaştırmalı Tablo

Özellik	Process	Thread
Bellek Yapısı	Ayrı adres uzayı	Ortak heap, ayrı stack
CPU Kullanımı	Paylaşılan (time slicing)	Paylaşılan (time slicing)
İzolasyon	Yüksek	Düşük
Context Switch Maliyeti	Yüksek	Düşük
Crash Etkisi	Diğerlerini etkilemez	Tüm process'i çökertir

Mobil Uygulama Bağlamında (Android / iOS)

Modern mobil uygulamalarda tipik olarak tek bir process bulunur ve bu process içinde birden fazla thread çalışır. Örneğin:

- UI Thread
- JS Thread
- Network Thread
- Image Decoding Thread
- Layout / Rendering Thread

Bu thread'lerin tamamı ortak bellek alanını kullanmakla birlikte, her biri bağımsız bir çağrı yığınına (stack) sahiptir.

JavaScript Event Loop Mekanizması

Bu bölüm, JavaScript çalışma zamanında (özellikle tarayıcı ve Node.js ortamlarında) asenkron yürütmeyi mümkün kılan *Event Loop* mekanizmasını resmi ve teknik bir dille açıklamaktadır.

Genel Tanım

JavaScript, tek iş parçacıklı (single-threaded) bir yürütme modeline sahiptir. Bu nedenle aynı anda yalnızca bir komut dizisi çalıştırabilir. Ancak, asenkron işlemler, zamanlayıcılar ve giriş/çıkış (I/O) operasyonları tarayıcı veya çalışma zamanı tarafından arka planda yürütülür. *Event Loop*, bu arka plan işlemlerinin sonuçlarını uygun sırayla JavaScript yürütme ortamına geri dahil eden kontrol mekanizmasıdır.

Temel Bileşenler

- **Call Stack (Çağrı Yığını):** JavaScript kodunun doğrudan yürütüldüğü yapıdır. Tek iş parçacıklı olduğundan, aynı anda yalnızca bir çerçeveye işlenir.
- **Web API'leri / Runtime API'leri:** Zamanlayıcılar (`setTimeout`, `setInterval`), ağ istekleri (`fetch`) ve diğer asenkron operasyonlar, JavaScript motoru dışında, tarayıcı veya Node.js tarafından gerçekleştirilir.
- **Task Queue (Macro Task Queue):** Zamanlayıcı callback'leri, DOM olayları ve diğer makro görevler, arka plandaki işlemler tamamlandığında bu kuyruğa eklenir.
- **Microtask Queue:** Özellikle `Promise` çözümleme callback'leri (`then/catch`) ve Mutation Observer görevleri burada tutulur. *Event Loop* her adımda öncelikle bu kuyruğu boşaltır.

Çalışma Prensipleri

Event Loop, sürekli olarak aşağıdaki döngüyü yürütür:

1. **Call Stack** boşalana kadar mevcut senkron kod yürütülür.
2. Call Stack boşaldığında, Event Loop önce **Microtask Queue**'yu kontrol eder. Eğer microtask varsa, tüm microtask'lar tamamlanana kadar Call Stack'e aktarılır.
3. Microtask Queue tamamen boşaldıktan sonra Event Loop, **Task Queue** içindeki ilk makro görevi Call Stack'e aktarır.
4. Bu döngü sonsuz şekilde devam eder.

Macro Task ve Micro Task Önceliği

Event Loop'un en önemli özelliği, microtask kuyruğunun her zaman makro görevlerden önce işlenmesidir. Bu nedenle, **Promise** tabanlı callback'ler, aynı döngü çevriminde **setTimeout** gibi zamanlayıcı görevlerinden önce yürütülür.

Örneğin:

```
console.log("A");  
Promise.resolve().then(() => console.log("B"));  
setTimeout(() => console.log("C"), 0);  
console.log("D");
```

Yürütme sırası:

$$A \rightarrow D \rightarrow B \rightarrow C$$

Özet

Event Loop, JavaScript'in tek iş parçacıklı yürütme modelini bloklamadan sürdürebilmesini sağlayan temel mekanizmadır. Arka planda tamamlanan asenkron operasyonların callback'leri, belirli kuyruklara eklenir ve Event Loop, Call Stack uygun olduğunda bu callback'leri yürütme sürecine dahil eder. Özellikle microtask kuyruğunun öncelikli olması, asenkron davranışın deterministik bir sıralamayla gerçekleşmesini sağlar.

Node.js'in Asenkron Mimarisinde Arka Plan İşlemlerinin Yönetimi

Bu bölüm, Node.js'in tek iş parçacıklı (single-threaded) JavaScript yürütme modeline rağmen asenkron görevleri nasıl arka planda gerçekleştirebildiğini resmi ve teknik bir dille açıklamaktadır. Node.js'in mimarisi, JavaScript motorunun tek iş parçacıklı yapısını korurken, I/O ve CPU ile ilgili birçok görevi çok iş parçacıklı bir altyapı ile yönetir.

Tek İş Parçacıklı JavaScript Yürütmesi

Node.js, JavaScript kodunu Google V8 motoru üzerinde yürütür ve bu yürütme tamamen *tek iş parçacıklıdır*. Yani:

- JavaScript kodu yalnızca bir *call stack* üzerinde çalışır.
- Her yürütme adımında yalnızca bir fonksiyon çerçevesi işlenebilir.
- Hesaplama yoğun görevler doğrudan bu iş parçacığını bloke edebilir.

Ancak bu sınırlama yalnızca JavaScript yürütme aşaması için geçerlidir. Node.js çekirdeğinde asenkron işlemleri yöneten farklı katmanlar bulunmaktadır.

libuv ve Çok İş Parçacıklı Altyapı

Node.js, asenkron davranışını sağlamak için **libuv** adlı bir C kütüphanesini kullanır. libuv, Node.js'e şu yetenekleri sağlar:

- Çok iş parçacıklı bir **Thread Pool** (varsayılan 4 iş parçacığı),
- Asenkron dosya işlemleri,
- DNS çözümleri,
- Zamanlayıcı yönetimi,
- Network polling mekanizmaları,
- OS tabanlı asenkron I/O arabirimleri.

Bu mimari sayesinde JavaScript kodu tek iş parçacıklı kalırken, I/O ve bazı CPU-bound işlemler farklı yürütme kaynaklarına aktarılır.

Arka Planda Gerçekleşen İşlemler

Node.js, uzun süren veya bloklayıcı olabilecek işlemleri doğrudan JavaScript iş parçacığı üzerinde çalıştırmaz. Bunun yerine:

1) İşletim Sistemi Asenkron I/O Mekanizmaları Ağ soketleri, çoğu dosya I/O işlemi ve OS destekli diğer asynchronous API'ler kernel tarafından asenkron olarak yürütülür. İşlem tamamlandığında libuv'a sinyal gönderilir ve ilgili callback *task queue*ya eklenir.

2) libuv Thread Pool Asenkron olmayan veya yüksek maliyetli görevler (örn. `fs` modülü, `crypto.pbkdf2`, `zlib` işlemleri) libuv'ın thread pool'u içinde çalıştırılır. Bu görevler JavaScript iş parçacığını bloke etmez.

Zamanlayıcılar

`setTimeout`, `setInterval` ve benzeri zamanlayıcılar JavaScript tarafından çalıştırılmaz. libuv, zamanlayıcıları kendi zamanlayıcı kuyruğunda takip eder. Süre dolduğunda, ilgili callback *ready queue*ya eklenir ve event loop tarafından çalıştırılır.

CPU-Bound Görevler İçin Çözümler

JavaScript iş parçacığını bloklamaması gereken yoğun CPU işlemleri için Node.js aşağıdaki mekanizmaları sağlar:

- **Worker Threads:** Gerçek çok iş parçacıklı hesaplama ortamı yaratır.
- **libuv Thread Pool:** Desteklenen API'ler otomatik olarak arka planda çalışır.
- **Child Process / Fork:** Ayrı bir Node.js süreci oluşturur.
- **Cluster:** Birden fazla Node.js sürecinin yük dengelemesi yapılır.

Mimari Özet

Aşağıdaki yapı, Node.js'in asenkron çalışmasını sağlayan katmanları göstermektedir:

- JavaScript Engine
(V8)
 - Tek iş parçacığı
 - Call Stack
 - Microtask Queue

- Event Loop
 - Görev zamanlama
 - Callback işleme

- libuv
 - Thread Pool (varsayılan 4 thread)
 - File I/O
 - DNS
 - Crypto
 - Timers
 - Network Polling

- Operating System APIs
(Asenkron socket ve disk I/O)

Özet

Node.js, JavaScript yürütmesini tek iş parçacıklı tutarken, asenkron I/O ve bazı CPU-bound görevleri libuv ve işletim sistemi aracılığıyla çok iş parçacıklı bir altyapıda gerçekleştirir. Bu mimari sayesinde Node.js yüksek eşzamanlılık sağlayabilir ve bloklanmadan çalışabilir.

Node.js'te Uzun Süreli Bir İşlemin Worker Thread ile Arka Plana Atılması

Aşağıda, CPU yoğun bir işlemin (örneğin büyük bir döngü) ana iş parçacığını bloklamadan başka bir iş parçacığına (**Worker Thread**) aktarılmasını gösteren minimal ve temiz bir Node.js örneği sunulmuştur. Bu yapı, uzun süren hesaplamaların event loop'u engellemeden yürütülmesini sağlar.

worker.js

```
// worker.js
const { parentPort } = require("worker_threads");

// CPU-yoğun örnek işlem
function heavyJob() {
  let total = 0;
  for (let i = 0; i < 300_000_000; i++) {
    total += i;
  }
  return total;
}

// Ana iş parçacığından mesaj alındığında işlemi başlat
parentPort.on("message", () => {
  const result = heavyJob();
  parentPort.postMessage(result);
});
```

main.js

```
// main.js
const { Worker } = require("worker_threads");

console.log("Ana thread çalışıyor...");

// Worker oluştur
const worker = new Worker("./worker.js");

// Worker'a görev gönder
worker.postMessage("start");

// Worker işlemi tamamladığında sonucu döndürür
worker.on("message", (result) => {
  console.log("Ağır işlem bitti. Sonuç:", result);
});
```

Açıklama

Bu örnekte:

- `heavyJob()` fonksiyonu CPU yoğun bir döngü içermektedir.
- Bu işlem doğrudan ana iş parçacığında çalıştırılsaydı event loop tamamen bloklanır ve diğer işlemler gecikirdi.
- `Worker` nesnesi aracılığıyla işlem ayrı bir iş parçacığında gerçekleştirilmekte ve sonuç `postMessage` ile geri bildirilmektedir.

Bu yaklaşım, Node.js uygulamalarında uzun hesaplamaların performans kaybına neden olmadan yürütülmesini mümkün kılar.

Node.js Worker Threads

Bu bölüm, **Worker Threads** özelliğinin Node.js çekirdeğinde gerçekten bulunan bir modül olup olmadığını teknik ve resmi bir dille açıklamaktadır.

Resmî Durum

Worker Threads, Node.js içerisinde yerleşik olarak bulunan, herhangi bir harici kütüphane gerektirmeyen, çekirdek (core) bir modüldür. Modül, aşağıdaki biçimde doğrudan **require** ifadesiyle kullanılabilir:

```
const { Worker } = require("worker_threads");
```

Sürüm Bilgisi

- Node.js 10.5 sürümü ile tanıtılmıştır.
- Node.js 12 ve üzeri sürümlerde varsayılan olarak etkindir.
- Herhangi bir ek kurulum, paket veya üçüncü parti bağımlılık gerektirmez.

Amaç ve Kullanım Alanı

Worker Threads modülü, JavaScript'in tek iş parçacıklı yürütme modelinin sınırlamalarını aşmak amacıyla geliştirilmiştir. Temel işlevi:

- CPU yoğun işlemleri ana iş parçacığını (event loop) bloklamadan yürütmek,
- Gerçek çok iş parçacıklı hesaplama olanağı sağlamak,
- **SharedArrayBuffer** ve **Atomics** ile paylaşılan bellek üzerinden paralel işlemeyi mümkün kılmak,
- Büyük döngüler, kriptografik hesaplamalar, sıkıştırma, makine öğrenmesi işlemleri gibi CPU-bound görevleri ayrı iş parçacıklarında çalıştırmaktır.

Resmî Doküman Referansı

Worker Threads modülü, Node.js'in resmî API dokümantasyonunda şu adreste belgelenmiştir:

https://nodejs.org/api/worker_threads.html

Özet

- **Worker Threads**, Node.js çekirdeğinde bulunan tamamen gerçek bir modüldür.
- Bu özellik, JavaScript yürütmesinden bağımsız olarak gerçek iş parçacıkları oluşturmak için kullanılır.
- Verilen örnek kodlar, Node.js ortamında doğrudan ve eksiksiz biçimde çalışır.
- Bu mekanizma, Node.js'in CPU yoğun işlemleri güvenli ve performanslı biçimde arka plana atabilmesini sağlar.

React Native’de Uzun Süreli İşlemlerin Arka Plana Atılması

React Native, JavaScript kodunu tek iş parçacığı üzerinde çalıştırır da, uzun süreli veya CPU yoğun işlemlerin kullanıcı arayüzünü (UI Thread) bloklamadan yürütülebilmesi için çeşitli mekanizmalar sunmaktadır. Bu bölümde React Native’de arka plan işlem yönetimi için kullanılan üç temel yaklaşım resmi ve teknik bir dille açıklanmıştır.

TurboModule / JSI ile Yerel (Native) İş Parçacıklarında İşlem

React Native 0.70+ mimarisi, JSI (JavaScript Interface) ve TurboModules üzerinden doğrudan C++ seviyesinde fonksiyon çağırmaı mümkün kılar. Bu yaklaşım, CPU yoğun görevlerin yerel iş parçacıklarında yürütülmesini sağlar.

C++ Tarafı (basitleştirilmiş örnek)

```
int heavyWork(int count) {
    long total = 0;
    for (int i = 0; i < count; i++) {
        total += i;
    }
    return total;
}
```

JavaScript Tarafında Çağırma

```
import { NativeModules } from 'react-native';

NativeModules.HeavyModule.heavyWork(300000000)
    .then(result => console.log(result));
```

Değerlendirme

- En yüksek performansı sunar.
- Gerçek native thread havuzları kullanılabilir.
- C++ bilgisi ve yapılandırma gerektirir.
- Üretim ortamı için en sağlam çözümdür.

InteractionManager ile UI Bloklamayan Gecikmeli İşler

InteractionManager gerçek bir iş parçacığı oluşturmaz; ancak animasyonlar ve kullanıcı etkileşimleri tamamlandıktan sonra ağır işlemlerin yürütülmesini planlamak için kullanılır.

```
import { InteractionManager } from 'react-native';

InteractionManager.runAfterInteractions(() => {
  // Uzun işlem burada yürütülür
});
```

Değerlendirme

- UI Thread'in anlık bloklanmasını önler.
- Ancak işlem yine JavaScript iş parçacığında çalışır.
- CPU yoğun işlemler için uygun değildir.

Özet

React Native doğrudan çok iş parçacıklı JavaScript yürütmesine sahip değildir. Ancak aşağıdaki mekanizmalarla uzun işlemler UI bloklamadan yürütülebilir:

- **react-native-threads:** Gerçek bağımsız JavaScript iş parçacıkları.
- **TurboModule + JSI:** Yerel iş parçacıklarında yüksek performanslı işlemler.
- **InteractionManager:** UI sonrası görev planlaması (thread değildir).

Bu yaklaşımlar, React Native uygulamalarında kullanıcı deneyimini bozmadan CPU yoğun işlemlerin yönetilmesini sağlar.

Sonuç

Mobil geliştirme ekosistemindeki performans ve kararlılık problemlerinin önemli bir kısmı, uygulamanın işletim sistemiyle nasıl etkileşime girdiğinin tam olarak anlaşılmasından kaynaklanmaktadır. Thread yönetimi, event loop davranışı, kullanıcı arayüzü ile arka plan görevlerinin ayrımı, bellek izolasyonu ve zamanlama mekanizmaları yalnızca düşük seviye teknik detaylar değil; modern mobil uygulamaların akıcı ve sorunsuz çalışması için temel yapı taşlarıdır.

Bu dokümanda ele alınan prensipler doğru uygulandığında:

- UI donmaları ve jank önemli ölçüde azalır,
- Ağır işlemler kullanıcı deneyimini etkilemeden arka planda yürütülebilir,
- React Native, Flutter ve native SDK'ların sınırları ve güçlü yönleri doğru yönetilir,
- Batarya tüketimi ve termal davranış daha verimli hale gelir,
- Uygulamaların genel kararlılığı ve kullanıcı memnuniyeti artar.

Mobil ekosistem gelişmeye devam etse de, işletim sistemi düzeyindeki bu temel prensipler kalıcıdır. Bu nedenle doküman, hem günlük geliştirme kararlarında hem de ileri seviye mimari tasarımlarda geliştiriciye yönlendirici bir referans niteliği taşımaktadır.